

Taller 1 – MountainCar con Q-Learning y DQN

Erwin Arnoldo Daza Rendón

Nicolás Rozo Hincapié

Fabián Eduardo Ruiz Sánchez

Andrés Enrique Pachón Bustos

Facultad de Ingeniería, Universidad de La Sabana

Simulación y Aprendizaje por Refuerzo

Dr. (c) Emilio Muñoz Pérez

19 de septiembre de 2026

TABLA DE CONTENIDO

I.	INTRODUCCIÓN.....	3
II.	PASO 1: REPOSITORIO EN GIT HUB	3
III.	PASO 2: ENTRENAMIENTO DEL AGENTE Q-LEARNING TABULAR	6
IV.	PASO 3: ENTRENAMIENTO DEL AGENTE DQN.....	18
V.	PASO 4: DOCUMENTACIÓN Y COMPARACIÓN DE RESULTADOS.....	34
VI.	REFERENCIAS.....	38

I. INTRODUCCIÓN

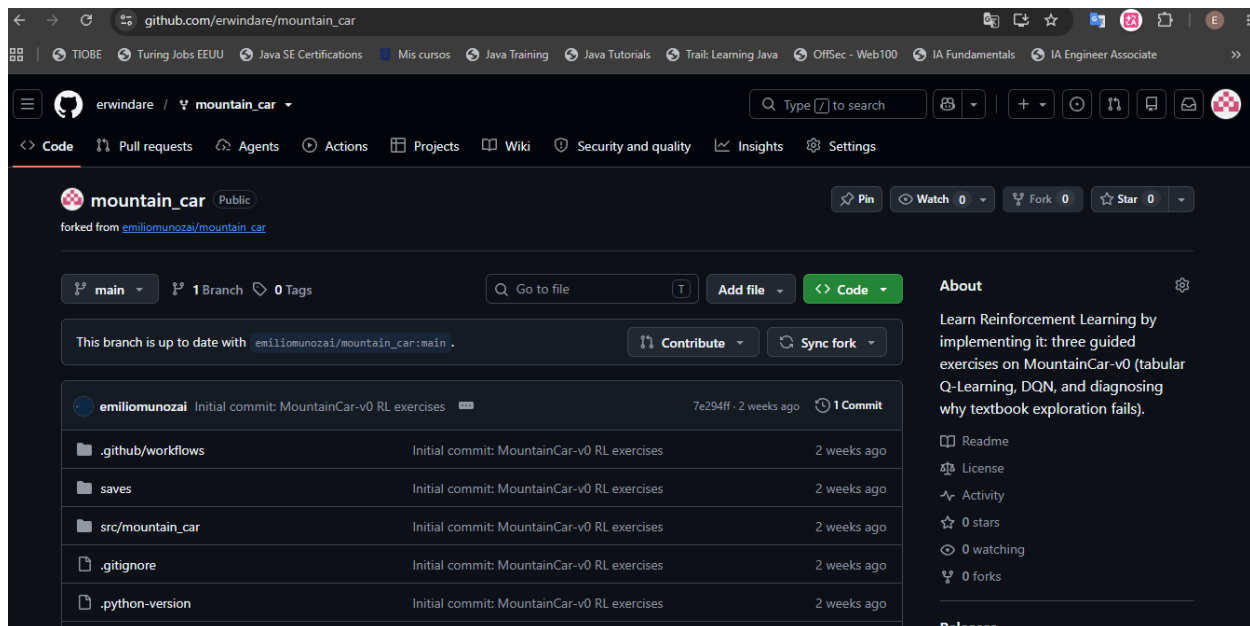
En esta actividad se implementarán y compararán dos enfoques de Aprendizaje por Refuerzo para resolver el entorno **MountainCar-v0**: un agente basado en **Q-Learning tabular**, mediante la discretización del espacio de estados, y un agente basado en **Deep Q-Network (DQN)**. El propósito es comprender las diferencias entre los métodos tabulares clásicos y los enfoques de Deep Reinforcement Learning, analizando el proceso de entrenamiento, el desempeño obtenido y las limitaciones de cada aproximación.

A continuación se mostrará el **paso a paso** del proceso seguido:

II. PASO 1: REPOSITORIO EN GIT HUB

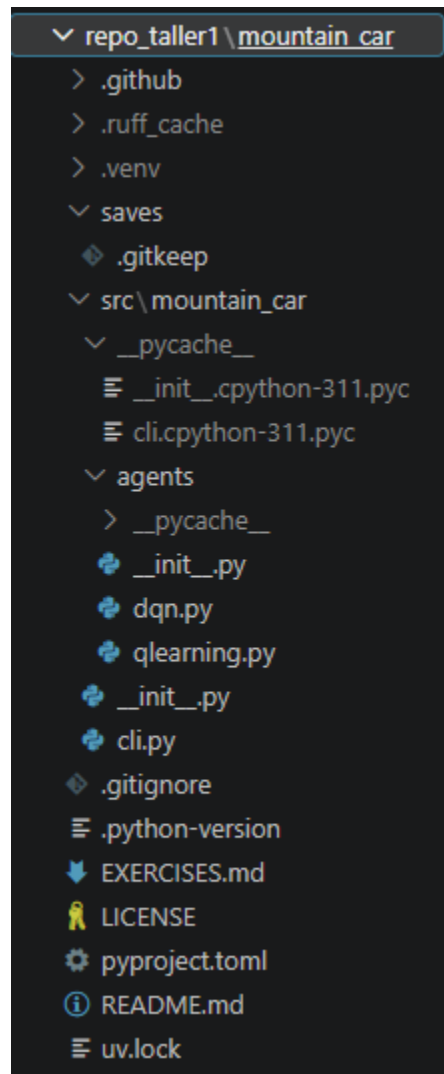
En este primer paso se procede a realizar una copia personal e independiente en la cuenta propia de GitHub, del repositorio base aportado en el curso. El repositorio base es: https://github.com/emiliomunozai/mountain_car. La copia de acceso Público se realiza en la cuenta de GitHub de Erwin Daza: https://github.com/erwindare/mountain_car como se puede evidenciar en la siguiente imagen:

Figura 1. Fork del repositorio base de GitHub



Una vez realizada esta copia se Clona localmente el proyecto para realizar los ajustes requeridos en el Taller. A continuación se evidencia la descarga del proyecto local y la estructura del mismo:

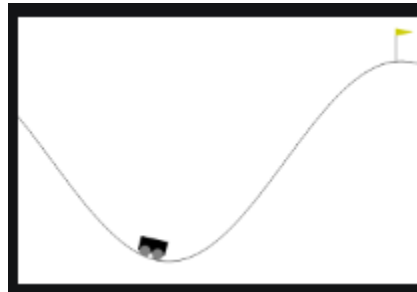
Figura 2. Clonación del proyecto



Ya teniendo todo dispuesto localmente, se procede a revisar los archivos **README.md** y **EXERCISES.md** para resolver los puntos del taller. Adicionalmente, se revisa la documentación de **Gymnasium** para ampliar el contexto del Entorno **MountainCar-v0** https://gymnasium.farama.org/environments/classic_control/mountain_car/.

El entorno de **Mountain Car** es un **Problema de Decisión de Markov (MDP)** determinista que consiste en un coche situado aleatoriamente en la parte inferior de un valle sinusoidal, donde las únicas acciones posibles son las aceleraciones que se le pueden aplicar en cualquier dirección. El objetivo del MDP es acelerar estratégicamente el coche para alcanzar el estado objetivo en la cima de la colina derecha. Veamos a continuación las características de este entorno:

Figura 3. *Mountain Car* (tomado de *Gymnasium*).



El **espacio de observación** de este MDP es una **ndarray** con forma **(2,)** donde los elementos corresponden a lo siguiente:

Tabla 1. *Espacio de observación*

Número	Observación	Min	Máximo	Unidad
0	Posición del coche a lo largo del eje X	-1.2	0.6	Posición (m)
1	Velocidad del coche	-0.07	0.07	Velocidad (v)

En el **espacio de acción** hay 3 acciones deterministas discretas:

- 0: Acelerar hacia la izquierda.
- 1: No acelera.
- 2: Acelerar hacia la derecha.

La **dinámica de transiciones** dice que ante una acción, el coche de montaña sigue las siguientes indicaciones:

$$\begin{aligned}
 \text{velocidad}_{t+1} &= \text{velocidad}_t + (\text{acción} - 1) * \text{fuerza} - \cos(3 * \text{posición}_t) * \text{gravedad} \\
 \text{posición}_{t+1} &= \text{posición}_t + \text{velocidad}_{t+1}
 \end{aligned}$$

Donde $\text{fuerza} = 0,001$ y $\text{gravedad} = 0,0025$. Las colisiones en ambos extremos son inelásticas con la velocidad establecida en 0 al colisionar con la pared. La posición está recortada al rango y la velocidad está recortada al rango: $[-1.2, 0.6]$ $[-0.07, 0.07]$.

El **premio** consiste en llegar a la bandera situada en la cima de la colina derecha lo más rápido posible; por ello, el agente es penalizado con una recompensa de -1 por cada paso de tiempo.

El **estado inicial** o **posición inicial** del coche se asigna a un valor aleatorio en $[-0,6, -0,4]$. La **velocidad** inicial del coche siempre se asigna a 0.

Un **episodio termina** si ocurre alguna de las siguientes situaciones:

- Finalización: La posición del coche es mayor o igual a 0.5 (la posición objetivo en la cima de la colina derecha).
- Recorte: La duración del episodio es de 200.

III. PASO 2: ENTRENAMIENTO DEL AGENTE Q-LEARNING TABULAR

En este paso se implementa los 3 ejercicios planteados en **EXERCISES.md** relacionados con el aprendizaje de Q-Tabular (archivo: **src/mountain_car/agents/qlearning.py**). Una vez implementados se procede a entrenar y evaluar el agente.

Primero se enumeran los agentes y se verifica si cada uno tiene su archivo correspondiente guardado. Esto se hace con el comando: **uv run mountaincar list**

```
• (ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar list
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Available agents:

  qlearning    [no save]  saves\qlearning_mountaincar.pkl
  dqn          [no save]  saves\dqn_mountaincar.pt
○ (ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> |
```

En el resultado del comando se evidencia que existen los agentes, pero ninguno tiene sus respectivos archivos guardados. En vista de esto se crea un agente nuevo para **Q-Learning**, sin entrenamiento y se guarda. Esto se hace con el comando: **uv run mountaincar init qlearning**

```
• (ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar init qlearning
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Saved Q-Learning agent to saves\qlearning_mountaincar.pkl
Initialized qlearning agent.
```

Al volver a listar vemos que se guarda el archivo **qlearning_mountaincar.pkl** para Q-Learning:

```
• (ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar list
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Available agents:

  qlearning    [saved]  saves\qlearning_mountaincar.pkl
  dqn          [no save]  saves\dqn_mountaincar.pt
```

A partir de este punto se procede a implementar los ejercicios planteados en **EXERCISES.md** para el agente **Q-Learning**:

Ejercicio 1: *Mapear una observación continua a una clave discreta de la tabla.* El retorno de este proceso será una **tupla** con un índice de intervalo por dimensión. Debido a que la **observación de MountainCar** son dos números continuos (posición, velocidad), se toma la estrategia de crear una **tabla Q** con sus claves discretas, tabla que se divide cada dimensión en ***n_bins*** intervalos y cada celda de la cuadrícula resultante será considerado como un estado.

De acuerdo a estas indicaciones, se propone la siguiente solución para discretizar:

```
def discretize(self, obs: np.ndarray) -> tuple:
    return tuple(int(np.digitize(value, edges)) for value, edges in zip(obs, self._bins))
```

En el código base se establecen en el constructor del agente **Q-Learning** el parámetro `n_bins = 20`, por lo que en la solución propuesta se crean 19 bordes internos, por tanto, los índices válidos son de 0 a 19. Un valor fuera del intervalo queda agrupado en el bin extremo más cercano, en lugar de generar un índice inválido. Esto no representa un problema en **MountainCar-v0**, porque las observaciones del entorno respetan los límites publicados:

- Posición: [-1.2, 0.6]
- Velocidad: [-0.07, 0.07]

Además, los valores exactamente iguales a los límites también se asignan correctamente a los bins extremos.

En resumen, la observación es solo bidimensional y el entorno publica límites estrictos para ambas dimensiones, por lo que el espacio de estados se discretiza en una `n_bins x n_bins` cuadrícula (400 estados por defecto) y se almacena en una tabla Q simple. Los valores predeterminados del agente son: `n_bins=20`, `lr=0.1`, `gamma=0.99`, `epsilon 1.0 -> 0.01` disminuye por 0.9995 episodio.

Ejercicio 2: *Selección de acciones mediante la estrategia epsilon-greedy.* En este punto la idea es desarrollar la política para el agente Q-Learning. La solución propuesta es la siguiente:

```
def select_action(self, state: tuple, *, deterministic: bool = False) -> int:
    if not deterministic and random.random() < self.epsilon:
        return random.randrange(self.n_actions)
    return int(np.argmax(self.q_table[state]))
```

En la función **select_action** implementamos la **política epsilon-greedy**, en donde definimos las siguientes reglas:

- Si **deterministic = false** y ocurre **exploración aleatoria** con **probabilidad epsilon menor a 100%**, se selecciona cualquiera de las siguientes acciones aleatorias:
 - 0: acelerar a la izquierda.
 - 1: no acelerar.
 - 2: acelerar a la derecha.
- En caso contrario, **explo**amos el conocimiento actual y seleccionamos la **acción con mayor valor Q** para el **estado discretizado** en el ejercicio anterior.

- Si el estado no existe en la tabla, se crea un vector [0, 0, 0]. En este caso, se devuelve inicialmente a la acción 0.
- Con **deterministic=true**, nunca explora y siempre utiliza la mejor acción conocida. Esto se usará durante evaluación y renderizado.

Ejercicio 3: *Actualización de Q-Learning*. La solución propuesta para este ejercicio es la siguiente:

```
def _update(
    self,
    state: tuple,
    action: int,
    reward: float,
    next_state: tuple,
    terminated: bool,
) -> None:
    target = reward if terminated else reward + self.gamma * np.max(self.q_table[next_state])
    self.q_table[state][action] += self.lr * (target - self.q_table[state][action])
```

En MountainCar la función **_update()** actualiza la tabla Q después de cada acción usando el algoritmo Q-Learning. Con una transición como esta:

estado → acción → recompensa → siguiente_estado (actualización)

En esta transición se tiene **objetivo “target”** es igual a:

$$\text{target} = \begin{cases} r, & \text{si el episodio terminó realmente} \\ r + \gamma \max_{a'} Q(s', a'), & \text{en otro caso} \end{cases}$$

Y luego se calcula el **valor de la acción** ejecutada: $Q(s,a) \leftarrow Q(s,a) + \alpha(\text{target} - Q(s,a))$

Q(s,a): es el valor estimado de ejecutar la acción **a** (*izquierda, quieto, derecha*) en el estado actual **s** (*posición y velocidad actuales discretizadas*).

Q(s',a'): es el valor estimado de ejecutar una posible acción **a'** (*una acción posible desde s'*) en el siguiente estado **s'** (*posición y velocidad después de ejecutar a*).

La actualización usa la mejor acción futura: **max Q(s', a')**.

En donde:

- **state**: estado discretizado actual.
- **action**: acción aplicada al coche.
- **reward**: normalmente -1 por avanzar un paso.

- **next_state**: siguiente estado discretizado.
- **terminated**: indica si el coche llegó a la bandera.
- **lr**: (α) tasa de aprendizaje.
- **gama**: (γ) importancia de recompensas futuras.

Una consideración importante a resaltar en esta solución de la “**actualización del nuevo estado**” es que en MountainCar:

- Si llega a la bandera, **terminated=true** y no se debe estimar ningún futuro.
- Si alcanza el límite de 200 pasos, **truncated=true**, pero el estado no es terminal. Por eso todavía se debe conservar el término futuro.

Una vez resueltos los tres ejercicios planteados para Q-Learning de acuerdo a las indicaciones dadas, se procede a **entrenar el agente con 20.000 episodios** con el siguiente comando: **uv run mountaincar train qlearning --episodes 20000**

Se crea el archivo binario del agente: **saves/qlearning_mountaincar.pkl** y se generan los 20.000 episodios:

```
Problems  Output  Debug Console  Terminal  Ports
Episode 18900/20000 | Avg Reward: -140.93 | Epsilon: 0.0100 | States visited: 297
Episode 19000/20000 | Avg Reward: -139.38 | Epsilon: 0.0100 | States visited: 297
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar train qlearning --episodes 20000
Episode 19200/20000 | Avg Reward: -139.69 | Epsilon: 0.0100 | States visited: 297
Episode 19300/20000 | Avg Reward: -134.19 | Epsilon: 0.0100 | States visited: 297
Episode 19400/20000 | Avg Reward: -140.09 | Epsilon: 0.0100 | States visited: 297
Episode 19500/20000 | Avg Reward: -138.91 | Epsilon: 0.0100 | States visited: 297
Episode 19600/20000 | Avg Reward: -134.24 | Epsilon: 0.0100 | States visited: 297
Episode 19700/20000 | Avg Reward: -142.48 | Epsilon: 0.0100 | States visited: 297
Episode 19800/20000 | Avg Reward: -136.11 | Epsilon: 0.0100 | States visited: 297
Episode 19900/20000 | Avg Reward: -136.99 | Epsilon: 0.0100 | States visited: 298
Episode 20000/20000 | Avg Reward: -140.03 | Epsilon: 0.0100 | States visited: 298
Saved Q-Learning agent to saves\qlearning_mountaincar.pkl
Training complete.
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car>
```

El detalle de los episodios es el siguiente:

Episode 100/20000	Avg Reward: -200.00	Epsilon: 0.9512	States visited: 74
Episode 200/20000	Avg Reward: -200.00	Epsilon: 0.9048	States visited: 77
Episode 300/20000	Avg Reward: -200.00	Epsilon: 0.8607	States visited: 95
Episode 400/20000	Avg Reward: -200.00	Epsilon: 0.8187	States visited: 113
Episode 500/20000	Avg Reward: -200.00	Epsilon: 0.7788	States visited: 115
Episode 600/20000	Avg Reward: -200.00	Epsilon: 0.7408	States visited: 129
Episode 700/20000	Avg Reward: -200.00	Epsilon: 0.7046	States visited: 152
Episode 800/20000	Avg Reward: -200.00	Epsilon: 0.6703	States visited: 161
Episode 900/20000	Avg Reward: -200.00	Epsilon: 0.6376	States visited: 172
Episode 1000/20000	Avg Reward: -200.00	Epsilon: 0.6065	States visited: 173
Episode 1100/20000	Avg Reward: -200.00	Epsilon: 0.5769	States visited: 206
Episode 1200/20000	Avg Reward: -200.00	Epsilon: 0.5487	States visited: 209

Episode 1300/20000	Avg Reward: -200.00	Epsilon: 0.5220	States visited: 217
Episode 1400/20000	Avg Reward: -200.00	Epsilon: 0.4965	States visited: 229
Episode 1500/20000	Avg Reward: -200.00	Epsilon: 0.4723	States visited: 236
Episode 1600/20000	Avg Reward: -200.00	Epsilon: 0.4492	States visited: 247
Episode 1700/20000	Avg Reward: -200.00	Epsilon: 0.4273	States visited: 247
Episode 1800/20000	Avg Reward: -200.00	Epsilon: 0.4065	States visited: 251
Episode 1900/20000	Avg Reward: -199.68	Epsilon: 0.3866	States visited: 257
Episode 2000/20000	Avg Reward: -199.62	Epsilon: 0.3678	States visited: 261
Episode 2100/20000	Avg Reward: -199.35	Epsilon: 0.3498	States visited: 266
Episode 2200/20000	Avg Reward: -198.54	Epsilon: 0.3328	States visited: 267
Episode 2300/20000	Avg Reward: -200.00	Epsilon: 0.3165	States visited: 267
Episode 2400/20000	Avg Reward: -195.50	Epsilon: 0.3011	States visited: 271
Episode 2500/20000	Avg Reward: -197.75	Epsilon: 0.2864	States visited: 277
Episode 2600/20000	Avg Reward: -193.31	Epsilon: 0.2724	States visited: 279
Episode 2700/20000	Avg Reward: -196.02	Epsilon: 0.2592	States visited: 281
Episode 2800/20000	Avg Reward: -198.49	Epsilon: 0.2465	States visited: 282
Episode 2900/20000	Avg Reward: -192.36	Epsilon: 0.2345	States visited: 282
Episode 3000/20000	Avg Reward: -186.46	Epsilon: 0.2230	States visited: 284
Episode 3100/20000	Avg Reward: -196.57	Epsilon: 0.2122	States visited: 286
Episode 3200/20000	Avg Reward: -188.64	Epsilon: 0.2018	States visited: 287
Episode 3300/20000	Avg Reward: -193.98	Epsilon: 0.1920	States visited: 287
Episode 3400/20000	Avg Reward: -183.44	Epsilon: 0.1826	States visited: 287
Episode 3500/20000	Avg Reward: -191.21	Epsilon: 0.1737	States visited: 287
Episode 3600/20000	Avg Reward: -193.89	Epsilon: 0.1652	States visited: 287
Episode 3700/20000	Avg Reward: -189.35	Epsilon: 0.1572	States visited: 287
Episode 3800/20000	Avg Reward: -180.49	Epsilon: 0.1495	States visited: 287
Episode 3900/20000	Avg Reward: -194.30	Epsilon: 0.1422	States visited: 288
Episode 4000/20000	Avg Reward: -194.02	Epsilon: 0.1353	States visited: 288
Episode 4100/20000	Avg Reward: -178.62	Epsilon: 0.1287	States visited: 288
Episode 4200/20000	Avg Reward: -178.79	Epsilon: 0.1224	States visited: 288
Episode 4300/20000	Avg Reward: -171.91	Epsilon: 0.1164	States visited: 290
Episode 4400/20000	Avg Reward: -180.85	Epsilon: 0.1107	States visited: 290
Episode 4500/20000	Avg Reward: -169.99	Epsilon: 0.1053	States visited: 291
Episode 4600/20000	Avg Reward: -173.42	Epsilon: 0.1002	States visited: 291
Episode 4700/20000	Avg Reward: -193.06	Epsilon: 0.0953	States visited: 291
Episode 4800/20000	Avg Reward: -194.85	Epsilon: 0.0907	States visited: 291
Episode 4900/20000	Avg Reward: -184.57	Epsilon: 0.0862	States visited: 291
Episode 5000/20000	Avg Reward: -185.46	Epsilon: 0.0820	States visited: 291
Episode 5100/20000	Avg Reward: -195.73	Epsilon: 0.0780	States visited: 291
Episode 5200/20000	Avg Reward: -175.37	Epsilon: 0.0742	States visited: 291
Episode 5300/20000	Avg Reward: -168.38	Epsilon: 0.0706	States visited: 291
Episode 5400/20000	Avg Reward: -167.61	Epsilon: 0.0672	States visited: 291
Episode 5500/20000	Avg Reward: -164.44	Epsilon: 0.0639	States visited: 291
Episode 5600/20000	Avg Reward: -171.11	Epsilon: 0.0608	States visited: 291
Episode 5700/20000	Avg Reward: -169.14	Epsilon: 0.0578	States visited: 291
Episode 5800/20000	Avg Reward: -171.47	Epsilon: 0.0550	States visited: 291

Episode 5900/20000	Avg Reward: -175.41	Epsilon: 0.0523	States visited: 291
Episode 6000/20000	Avg Reward: -173.74	Epsilon: 0.0497	States visited: 291
Episode 6100/20000	Avg Reward: -179.00	Epsilon: 0.0473	States visited: 291
Episode 6200/20000	Avg Reward: -176.18	Epsilon: 0.0450	States visited: 291
Episode 6300/20000	Avg Reward: -154.98	Epsilon: 0.0428	States visited: 292
Episode 6400/20000	Avg Reward: -170.53	Epsilon: 0.0407	States visited: 292
Episode 6500/20000	Avg Reward: -171.20	Epsilon: 0.0387	States visited: 292
Episode 6600/20000	Avg Reward: -157.57	Epsilon: 0.0369	States visited: 292
Episode 6700/20000	Avg Reward: -172.70	Epsilon: 0.0351	States visited: 292
Episode 6800/20000	Avg Reward: -167.26	Epsilon: 0.0333	States visited: 293
Episode 6900/20000	Avg Reward: -165.58	Epsilon: 0.0317	States visited: 293
Episode 7000/20000	Avg Reward: -165.81	Epsilon: 0.0302	States visited: 293
Episode 7100/20000	Avg Reward: -162.72	Epsilon: 0.0287	States visited: 293
Episode 7200/20000	Avg Reward: -153.31	Epsilon: 0.0273	States visited: 293
Episode 7300/20000	Avg Reward: -162.46	Epsilon: 0.0260	States visited: 293
Episode 7400/20000	Avg Reward: -177.57	Epsilon: 0.0247	States visited: 293
Episode 7500/20000	Avg Reward: -159.40	Epsilon: 0.0235	States visited: 293
Episode 7600/20000	Avg Reward: -171.58	Epsilon: 0.0223	States visited: 293
Episode 7700/20000	Avg Reward: -171.74	Epsilon: 0.0213	States visited: 293
Episode 7800/20000	Avg Reward: -176.02	Epsilon: 0.0202	States visited: 293
Episode 7900/20000	Avg Reward: -151.95	Epsilon: 0.0192	States visited: 293
Episode 8000/20000	Avg Reward: -147.80	Epsilon: 0.0183	States visited: 293
Episode 8100/20000	Avg Reward: -155.08	Epsilon: 0.0174	States visited: 293
Episode 8200/20000	Avg Reward: -157.43	Epsilon: 0.0166	States visited: 293
Episode 8300/20000	Avg Reward: -153.62	Epsilon: 0.0157	States visited: 293
Episode 8400/20000	Avg Reward: -161.50	Epsilon: 0.0150	States visited: 293
Episode 8500/20000	Avg Reward: -164.48	Epsilon: 0.0142	States visited: 293
Episode 8600/20000	Avg Reward: -156.10	Epsilon: 0.0136	States visited: 293
Episode 8700/20000	Avg Reward: -145.19	Epsilon: 0.0129	States visited: 293
Episode 8800/20000	Avg Reward: -159.09	Epsilon: 0.0123	States visited: 293
Episode 8900/20000	Avg Reward: -148.80	Epsilon: 0.0117	States visited: 293
Episode 9000/20000	Avg Reward: -140.67	Epsilon: 0.0111	States visited: 293
Episode 9100/20000	Avg Reward: -152.23	Epsilon: 0.0106	States visited: 293
Episode 9200/20000	Avg Reward: -162.69	Epsilon: 0.0100	States visited: 293
Episode 9300/20000	Avg Reward: -160.75	Epsilon: 0.0100	States visited: 293
Episode 9400/20000	Avg Reward: -167.45	Epsilon: 0.0100	States visited: 293
Episode 9500/20000	Avg Reward: -160.35	Epsilon: 0.0100	States visited: 293
Episode 9600/20000	Avg Reward: -167.22	Epsilon: 0.0100	States visited: 293
Episode 9700/20000	Avg Reward: -162.56	Epsilon: 0.0100	States visited: 293
Episode 9800/20000	Avg Reward: -164.16	Epsilon: 0.0100	States visited: 293
Episode 9900/20000	Avg Reward: -178.06	Epsilon: 0.0100	States visited: 293
Episode 10000/20000	Avg Reward: -158.42	Epsilon: 0.0100	States visited: 293
Episode 10100/20000	Avg Reward: -151.84	Epsilon: 0.0100	States visited: 295
Episode 10200/20000	Avg Reward: -143.55	Epsilon: 0.0100	States visited: 295
Episode 10300/20000	Avg Reward: -140.72	Epsilon: 0.0100	States visited: 295
Episode 10400/20000	Avg Reward: -142.27	Epsilon: 0.0100	States visited: 295

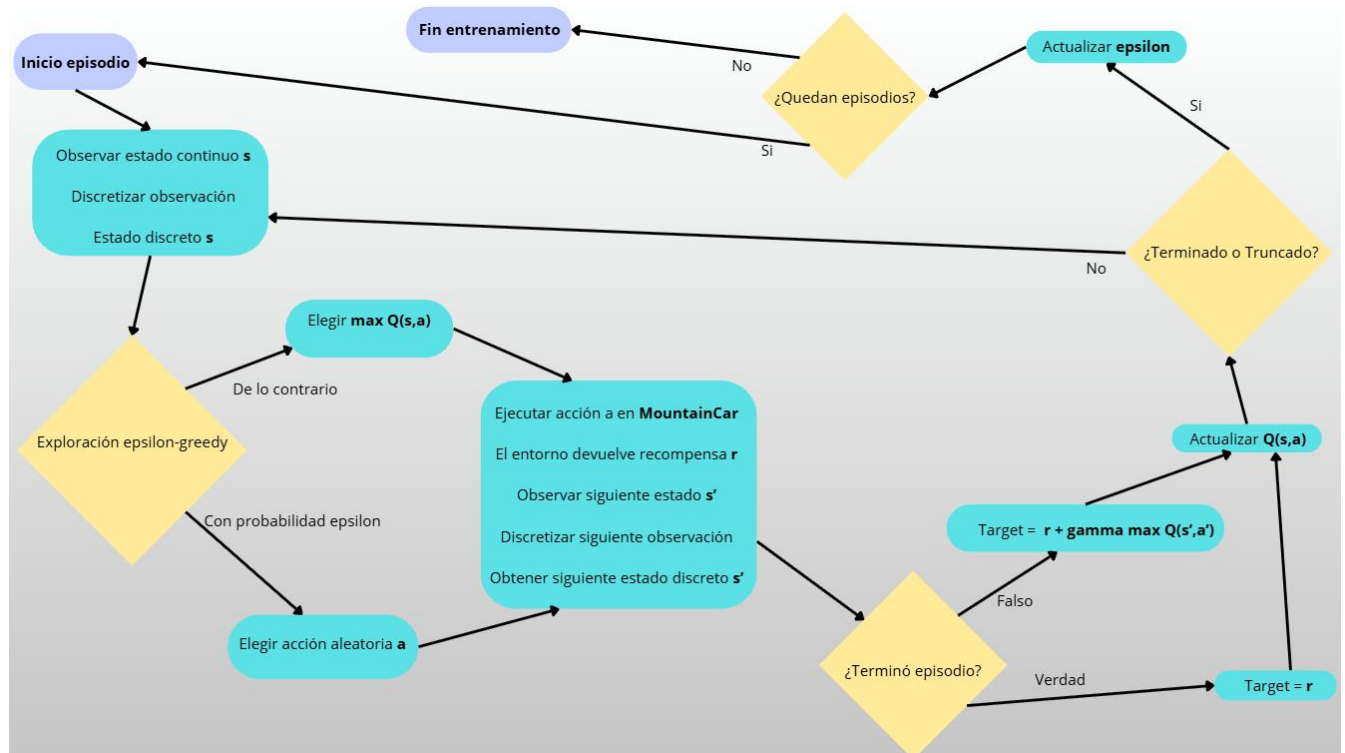
Episode 10500/20000	Avg Reward: -137.46	Epsilon: 0.0100	States visited: 295
Episode 10600/20000	Avg Reward: -144.47	Epsilon: 0.0100	States visited: 295
Episode 10700/20000	Avg Reward: -149.43	Epsilon: 0.0100	States visited: 295
Episode 10800/20000	Avg Reward: -146.03	Epsilon: 0.0100	States visited: 295
Episode 10900/20000	Avg Reward: -145.12	Epsilon: 0.0100	States visited: 295
Episode 11000/20000	Avg Reward: -150.95	Epsilon: 0.0100	States visited: 295
Episode 11100/20000	Avg Reward: -147.76	Epsilon: 0.0100	States visited: 295
Episode 11200/20000	Avg Reward: -150.80	Epsilon: 0.0100	States visited: 295
Episode 11300/20000	Avg Reward: -151.29	Epsilon: 0.0100	States visited: 295
Episode 11400/20000	Avg Reward: -146.76	Epsilon: 0.0100	States visited: 295
Episode 11500/20000	Avg Reward: -141.83	Epsilon: 0.0100	States visited: 295
Episode 11600/20000	Avg Reward: -148.09	Epsilon: 0.0100	States visited: 295
Episode 11700/20000	Avg Reward: -147.88	Epsilon: 0.0100	States visited: 295
Episode 11800/20000	Avg Reward: -146.78	Epsilon: 0.0100	States visited: 295
Episode 11900/20000	Avg Reward: -158.40	Epsilon: 0.0100	States visited: 295
Episode 12000/20000	Avg Reward: -161.63	Epsilon: 0.0100	States visited: 295
Episode 12100/20000	Avg Reward: -162.59	Epsilon: 0.0100	States visited: 295
Episode 12200/20000	Avg Reward: -164.22	Epsilon: 0.0100	States visited: 295
Episode 12300/20000	Avg Reward: -167.16	Epsilon: 0.0100	States visited: 295
Episode 12400/20000	Avg Reward: -165.09	Epsilon: 0.0100	States visited: 295
Episode 12500/20000	Avg Reward: -150.09	Epsilon: 0.0100	States visited: 295
Episode 12600/20000	Avg Reward: -155.37	Epsilon: 0.0100	States visited: 295
Episode 12700/20000	Avg Reward: -146.73	Epsilon: 0.0100	States visited: 296
Episode 12800/20000	Avg Reward: -150.92	Epsilon: 0.0100	States visited: 296
Episode 12900/20000	Avg Reward: -145.28	Epsilon: 0.0100	States visited: 296
Episode 13000/20000	Avg Reward: -155.73	Epsilon: 0.0100	States visited: 296
Episode 13100/20000	Avg Reward: -166.81	Epsilon: 0.0100	States visited: 297
Episode 13200/20000	Avg Reward: -156.67	Epsilon: 0.0100	States visited: 297
Episode 13300/20000	Avg Reward: -145.78	Epsilon: 0.0100	States visited: 297
Episode 13400/20000	Avg Reward: -148.71	Epsilon: 0.0100	States visited: 297
Episode 13500/20000	Avg Reward: -136.07	Epsilon: 0.0100	States visited: 297
Episode 13600/20000	Avg Reward: -141.42	Epsilon: 0.0100	States visited: 297
Episode 13700/20000	Avg Reward: -139.25	Epsilon: 0.0100	States visited: 297
Episode 13800/20000	Avg Reward: -141.17	Epsilon: 0.0100	States visited: 297
Episode 13900/20000	Avg Reward: -146.54	Epsilon: 0.0100	States visited: 297
Episode 14000/20000	Avg Reward: -147.40	Epsilon: 0.0100	States visited: 297
Episode 14100/20000	Avg Reward: -161.90	Epsilon: 0.0100	States visited: 297
Episode 14200/20000	Avg Reward: -163.02	Epsilon: 0.0100	States visited: 297
Episode 14300/20000	Avg Reward: -162.67	Epsilon: 0.0100	States visited: 297
Episode 14400/20000	Avg Reward: -158.17	Epsilon: 0.0100	States visited: 297
Episode 14500/20000	Avg Reward: -153.03	Epsilon: 0.0100	States visited: 297
Episode 14600/20000	Avg Reward: -145.84	Epsilon: 0.0100	States visited: 297
Episode 14700/20000	Avg Reward: -148.43	Epsilon: 0.0100	States visited: 297
Episode 14800/20000	Avg Reward: -144.50	Epsilon: 0.0100	States visited: 297
Episode 14900/20000	Avg Reward: -144.24	Epsilon: 0.0100	States visited: 297
Episode 15000/20000	Avg Reward: -140.35	Epsilon: 0.0100	States visited: 297

Episode 15100/20000	Avg Reward: -141.78	Epsilon: 0.0100	States visited: 297
Episode 15200/20000	Avg Reward: -144.42	Epsilon: 0.0100	States visited: 297
Episode 15300/20000	Avg Reward: -144.30	Epsilon: 0.0100	States visited: 297
Episode 15400/20000	Avg Reward: -144.30	Epsilon: 0.0100	States visited: 297
Episode 15500/20000	Avg Reward: -149.19	Epsilon: 0.0100	States visited: 297
Episode 15600/20000	Avg Reward: -149.12	Epsilon: 0.0100	States visited: 297
Episode 15700/20000	Avg Reward: -149.81	Epsilon: 0.0100	States visited: 297
Episode 15800/20000	Avg Reward: -146.36	Epsilon: 0.0100	States visited: 297
Episode 15900/20000	Avg Reward: -148.53	Epsilon: 0.0100	States visited: 297
Episode 16000/20000	Avg Reward: -150.97	Epsilon: 0.0100	States visited: 297
Episode 16100/20000	Avg Reward: -147.92	Epsilon: 0.0100	States visited: 297
Episode 16200/20000	Avg Reward: -152.24	Epsilon: 0.0100	States visited: 297
Episode 16300/20000	Avg Reward: -166.07	Epsilon: 0.0100	States visited: 297
Episode 16400/20000	Avg Reward: -169.90	Epsilon: 0.0100	States visited: 297
Episode 16500/20000	Avg Reward: -167.76	Epsilon: 0.0100	States visited: 297
Episode 16600/20000	Avg Reward: -154.37	Epsilon: 0.0100	States visited: 297
Episode 16700/20000	Avg Reward: -154.67	Epsilon: 0.0100	States visited: 297
Episode 16800/20000	Avg Reward: -150.11	Epsilon: 0.0100	States visited: 297
Episode 16900/20000	Avg Reward: -152.46	Epsilon: 0.0100	States visited: 297
Episode 17000/20000	Avg Reward: -150.01	Epsilon: 0.0100	States visited: 297
Episode 17100/20000	Avg Reward: -149.13	Epsilon: 0.0100	States visited: 297
Episode 17200/20000	Avg Reward: -149.15	Epsilon: 0.0100	States visited: 297
Episode 17300/20000	Avg Reward: -149.97	Epsilon: 0.0100	States visited: 297
Episode 17400/20000	Avg Reward: -150.53	Epsilon: 0.0100	States visited: 297
Episode 17500/20000	Avg Reward: -150.45	Epsilon: 0.0100	States visited: 297
Episode 17600/20000	Avg Reward: -151.03	Epsilon: 0.0100	States visited: 297
Episode 17700/20000	Avg Reward: -157.23	Epsilon: 0.0100	States visited: 297
Episode 17800/20000	Avg Reward: -150.22	Epsilon: 0.0100	States visited: 297
Episode 17900/20000	Avg Reward: -150.22	Epsilon: 0.0100	States visited: 297
Episode 18000/20000	Avg Reward: -152.03	Epsilon: 0.0100	States visited: 297
Episode 18100/20000	Avg Reward: -155.18	Epsilon: 0.0100	States visited: 297
Episode 18200/20000	Avg Reward: -161.22	Epsilon: 0.0100	States visited: 297
Episode 18300/20000	Avg Reward: -155.60	Epsilon: 0.0100	States visited: 297
Episode 18400/20000	Avg Reward: -141.08	Epsilon: 0.0100	States visited: 297
Episode 18500/20000	Avg Reward: -153.56	Epsilon: 0.0100	States visited: 297
Episode 18600/20000	Avg Reward: -156.01	Epsilon: 0.0100	States visited: 297
Episode 18700/20000	Avg Reward: -133.83	Epsilon: 0.0100	States visited: 297
Episode 18800/20000	Avg Reward: -144.18	Epsilon: 0.0100	States visited: 297
Episode 18900/20000	Avg Reward: -140.93	Epsilon: 0.0100	States visited: 297
Episode 19000/20000	Avg Reward: -139.28	Epsilon: 0.0100	States visited: 297
Episode 19100/20000	Avg Reward: -140.87	Epsilon: 0.0100	States visited: 297
Episode 19200/20000	Avg Reward: -139.69	Epsilon: 0.0100	States visited: 297
Episode 19300/20000	Avg Reward: -134.19	Epsilon: 0.0100	States visited: 297
Episode 19400/20000	Avg Reward: -140.09	Epsilon: 0.0100	States visited: 297
Episode 19500/20000	Avg Reward: -138.91	Epsilon: 0.0100	States visited: 297
Episode 19600/20000	Avg Reward: -134.24	Epsilon: 0.0100	States visited: 297

Episode 19700/20000 | Avg Reward: -142.48 | Epsilon: 0.0100 | States visited: 297
 Episode 19800/20000 | Avg Reward: -136.11 | Epsilon: 0.0100 | States visited: 297
 Episode 19900/20000 | Avg Reward: -136.99 | Epsilon: 0.0100 | States visited: 298
 Episode 20000/20000 | Avg Reward: -140.03 | Epsilon: 0.0100 | States visited: 298

Cada ciclo de entrenamiento dado por los episodios pueden entenderse de la siguiente manera:

Figura 4. Diagrama de pasos por cada episodio en el entrenamiento del agente Q-Learning



Ahora se procede a evaluar y a imprimir la información consolidada del resultado obtenido con el comando: **`uv run mountaincar load qlearning --eval`**

```

Q-Learning agent for MountainCar-v0
Episodes trained : 20000
States visited   : 298 / 400
Epsilon          : 0.0100
LR / Gamma      : 0.1 / 0.99

Evaluating (10 episodes) ...
Mean reward: -141.60 +/- 23.52
Reached the flag: 10/10 episodes
    
```

A partir de los resultados obtenidos en el entrenamiento y en la evaluación, se evidencia que la Recompensa Promedio inicialmente se mantiene estable en -200 y es cuando se visitan la menor

cantidad de estados. Esto pasa durante un tiempo y luego esta Recompensa empieza a subir a medida que se van visitando más estados.

En conclusión, los resultados muestran que el agente aprendió una política funcional para MountainCar-v0:

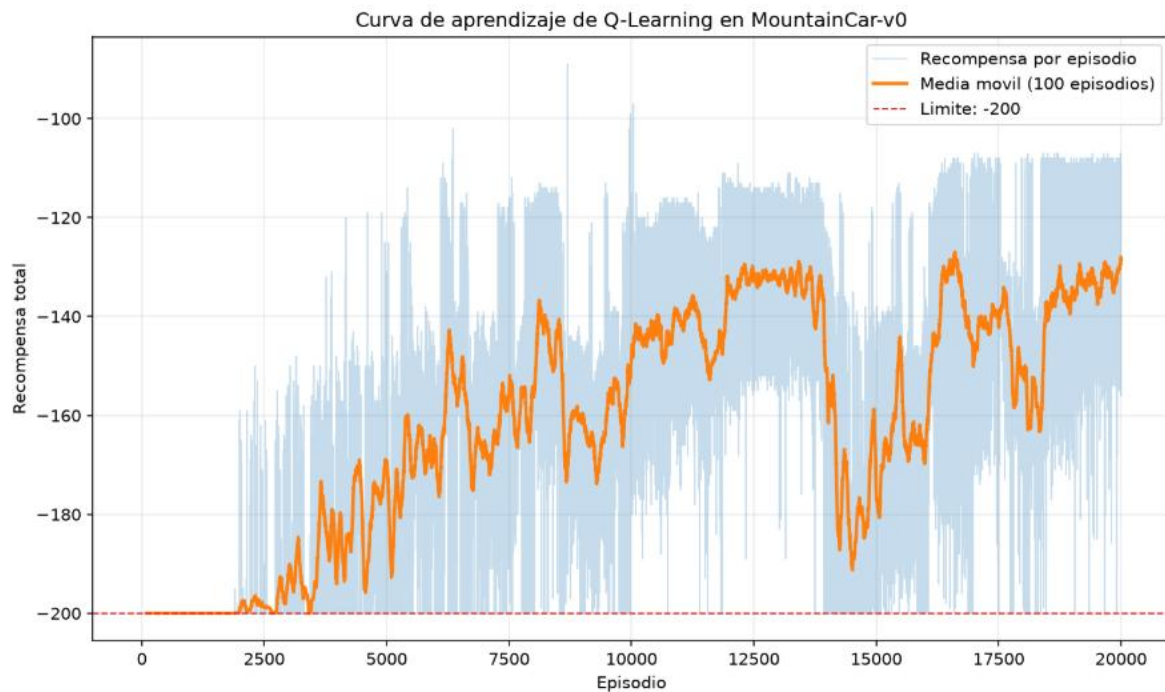
- 20.000 episodios entrenados: tuvo suficientes interacciones para explorar y actualizar la tabla Q.
- 298 de 400 estados visitados: exploró aproximadamente el 74,5 % del espacio discretizado. No es necesario visitar todos los estados para resolver el problema, porque muchos son poco relevantes o inalcanzables durante una trayectoria útil.
- Epsilon = 0.01: durante la evaluación, el agente utiliza casi siempre la mejor acción conocida. La exploración quedó reducida al 1 %.
- Mean reward = -141.60: cada paso cuesta -1, por lo que el agente tarda aproximadamente 142 pasos en alcanzar la bandera. Es mejor que el límite de -200, que representa no resolver el episodio.
- Desviación estándar = 23.52: existe variabilidad entre episodios. Algunas trayectorias alcanzan la bandera rápidamente y otras requieren más oscilaciones para tomar impulso.
- Reached the flag = 10/10: es el resultado más importante. El agente logra llegar a la posición objetivo en todos los episodios evaluados.

El agente aprendió la estrategia esencial de MountainCar: mover el coche hacia ambos lados para ganar velocidad y luego alcanzar la colina derecha. Aunque el promedio -141.60 todavía puede mejorar, el agente ya resolvió consistentemente la tarea. La tabla Q no necesita representar perfectamente todos los estados; basta con aprender correctamente los estados y acciones relevantes para llegar a la bandera.

A continuación vemos la **curva de aprendizaje** del agente **Q-Learning** a través de los 20.000 episodios, generada con el siguiente comando ejecutado desde la carpeta **mountain_car**:

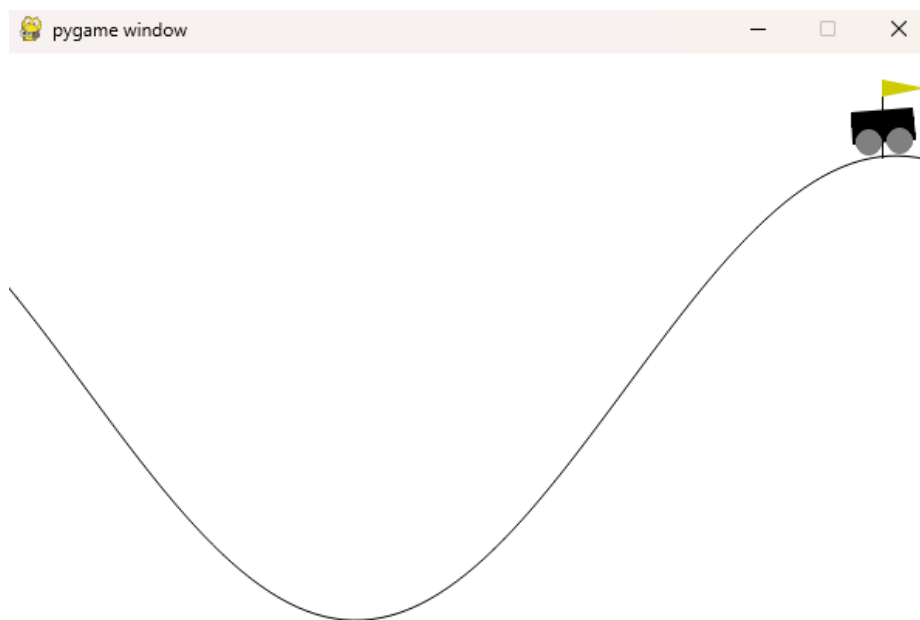
```
.\.venv\Scripts\python.exe .\graficos\graficas_qlearning.py --episodes 20000 --window 100
```

Figura 5. *Curva de aprendizaje del agente Q-Learning.*



En la gráfica anterior se puede ver el resumen del comportamiento del entrenamiento, en donde inicialmente el agente no aprende, se mantiene en -200 y posteriormente cuando explora más estados su aprendizaje va mejorando, **hasta alcanzar recompensas medias alrededor de -130**.

Finalmente, vamos a reproducir episodios en una ventana gráfica a través del siguiente comando: ***uv run mountaincar render qlearning --episodes 3***



```
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar render qlearning --episodes 3
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Built mountain-car @ file:///D:/MaestriaEnIA/Semestre2/Ciclo2/Code/RL/repo_taller1/mountain_car
Uninstalled 1 package in 17ms
[0/1] Installing wheels...
warning: Failed to hardlink files; falling back to full copy. This may lead to degraded performance.
If the cache and target directories are on different filesystems, hardlinking may not be supported.
If this is intentional, set `export UV_LINK_MODE=copy` or use `--link-mode=copy` to suppress this warning.
Installed 1 package in 33ms
Episode 1/3 | Reward: -162.00 | Steps: 162 | reached the flag
Episode 2/3 | Reward: -159.00 | Steps: 159 | reached the flag
Episode 3/3 | Reward: -162.00 | Steps: 162 | reached the flag
```

En la gráfica anterior, se puede observar cómo el carrito logra llegar al objetivo (la bandera en la cima de la colina derecha) en los tres episodios de prueba.

ESCENARIOS CON HIPERPARÁMETROS DISTINTOS

En el ejercicio de Q-Learning se decide hacer un análisis adicional para evaluar diferentes escenarios variando los hiperparámetros y validar que tanto impacto tienen sobre el resultado. El propósito principal. Se evalúan las siguientes variaciones:

Escenario	n_bins	lr	gamma	epsilon_decay	Episodios	Objetivo
Base	20	0.10	0.99	0.9995	20.000	Referencia
Mayor estabilidad	20	0.05	0.99	0.9995	20.000	Reducir fluctuaciones
Más exploración	20	0.10	0.99	0.9997	20.000	Explorar durante más tiempo
Mayor resolución	30	0.10	0.99	0.9995	20.000	Representar mejor posición/velocidad

Las conclusiones de cada escenario a continuación:

i. Escenario de Mayor Estabilidad

La configuración inicial con $lr=0,1$ permitió que el agente alcanzara la meta en los 10 episodios de evaluación, con una recompensa promedio de $-149,10 \pm 11,41$. Al reducir la tasa de aprendizaje a 0,05, el agente visitó 297 de los 400 estados posibles, frente a 292 estados en la configuración inicial. Durante el entrenamiento también se observaron periodos de rendimiento notablemente superiores, alcanzando recompensas promedio cercanas a -113.

Estos resultados sugieren que una tasa de aprendizaje menor puede favorecer actualizaciones más graduales de la Q-table y permitir la construcción de políticas más eficientes en determinados momentos del entrenamiento.

ii. Escenario de Mayor exploración

Al modificar `epsilon_decay` de 0,9995 a 0,9997, se prolongó considerablemente la fase de exploración. Epsilon alcanzó su valor mínimo aproximadamente alrededor del episodio 15.400, en lugar de hacerlo cerca del episodio 9.200. Esta configuración también alcanzó 297 estados visitados y finalizó con una recompensa promedio de -145,36. El principal efecto observado fue una exploración más prolongada y un aprendizaje más gradual, aunque sin alcanzar durante el entrenamiento los valores máximos obtenidos con $\text{lr}=0,05$.

iii. Escenario de Mayor resolución

El cambio con mayor impacto en la evaluación determinística fue el aumento de la discretización a 30×30 , generando 900 estados posibles. En este escenario el agente visitó 665 estados y alcanzó la bandera en 10 de 10 episodios, con una recompensa promedio de $-131,30 \pm 13,21$. Frente a la configuración inicial, esto representa una mejora de 17,8 puntos en la recompensa promedio y aproximadamente un 11,9 % de reducción en los pasos necesarios para alcanzar la meta.

Los resultados sugieren que una discretización más detallada permitió representar con mayor precisión las combinaciones de posición y velocidad del vehículo, facilitando la selección de acciones más adecuadas en diferentes regiones del espacio de estados. Sin embargo, también incrementó el tamaño de la Q-table y, por tanto, la cantidad de estados que el agente debe explorar.

Entre las configuraciones evaluadas de forma determinística, `n_bins=30` presentó el mejor desempeño, manteniendo una tasa de éxito del 100 % y obteniendo una recompensa promedio de -131,30, superior al resultado de la configuración inicial

IV. PASO 3: ENTRENAMIENTO DEL AGENTE DQN

En este paso se implementa los 3 ejercicios planteados en **EXERCISES.md** relacionados con el aprendizaje de Red Q Profunda – DQN (archivo: `src/mountain_car/agents/dqn.py`). Una vez implementados se procede a entrenar y evaluar el agente. El algoritmo a desarrollar es el mismo que el del agente Q-Learning, pero **la tabla Q se convierte en una red neuronal**, lo que significa que no hay discretización y se necesitan dos máquinas nuevas: un búfer de reproducción (dado) y una red objetivo (dada).

En este punto verificaremos que el agente DQN no tiene archivo binario guardado. Esto lo hacemos con el comando: ***uv run mountaincar list***

```
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar list
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active e
nvironment instead
Available agents:

qlearning      [saved]  saves\qlearning_mountaincar.pkl
dqn            [no save] saves\dqn_mountaincar.pt
```

En vista de ello, procedemos a implementar los ejercicios planteados en **EXERCISES.md** para el agente **Red Q Profunda**:

Ejercicio 1: *Red que mapea un estado a un valor Q por acción.* Se debe construir una red pequeña conectada con la siguiente estructura:

state_dim → hidden → hidden → action_dim

Con una función ReLU en cada capa oculta. No hay activación en la capa de salida: Se trata de valores Q (todos son negativos). De acuerdo a estas indicaciones, se propone la siguiente solución para construir la Red solicitada:

```
class QNetwork(nn.Module):

    def __init__(self, state_dim: int, action_dim: int, hidden: int = 128) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.ReLU(),
            nn.Linear(hidden, action_dim),
        )

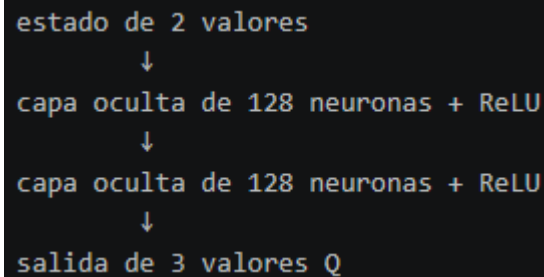
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

El código anterior representa una red neuronal que recibe el estado continuo de MountainCar y produce un valor Q por cada acción; además, el agente crea una red online y otra target con la misma arquitectura. Con esto se busca estimar que tan conveniente es ejecutar cada acción desde un estado determinado. La configuración de la red sería la siguiente:

- **state_dim:** cantidad de variables del estado. En MountainCar son 2 (posición, velocidad).
- **action_dim:** cantidad de acciones posibles. En MountainCar son 3 (izquierda, no acelera, derecha).
- **hidden:** número de neuronas en cada capa oculta, por defecto se establecen 128.
- **super().__init__():** inicializa correctamente la clase base.

- **self.net**: crea la arquitectura.

Lo anterior, podemos resumirlo de la siguiente manera:



La salida tiene tres valores Q, uno por acción. En cuanto a **forward()**, define cómo fluye la información por la red. Recibe uno o varios estados y los pasa por las capas de **self.net**.

Ejercicio 2: *Paso de aprendizaje por DQN*. En este punto el mini-lote ya está muestreado y convertido a tensores. En esta parte se deben hacer las siguientes cosas:

- Mantener un registro de las formas del tensor . `self.q_net(states_t)` es (batch, action_dim), pero la pérdida compara un número por transición, por lo que ambos lados deberían terminar (batch, 1).
- `next_q` debe provenir de `self.target_net`, no de `self.q_net`, y no debe contener gradientes.
- Leer el bucle de entrenamiento y observa qué se envía al búfer de reproducción como la bandera de "terminado". Es `terminated`, no `terminated` or `truncated`.
- Realiza un paso de gradiente sobre `self.loss_fn(current_q, target_q)`.

Para estas indicaciones, la solución propuesta es la siguiente:

```
class DQNAgent:

    def _learn(self) -> float:
        """Sample a mini-batch from the buffer and take one gradient step.

        Returns the batch loss value.
        """
        if len(self.buffer) < self.batch_size:
            return 0.0

        batch = self.buffer.sample(self.batch_size)
        states, actions, rewards, next_states, terminateds = zip(*batch)
        states_t = self._tensor(states)
        actions_t = self._tensor(actions, torch.int64).unsqueeze(1)
        rewards_t = self._tensor(rewards).unsqueeze(1)
        next_states_t = self._tensor(next_states)
        terminateds_t = self._tensor(terminateds).unsqueeze(1)

        current_q = self.q_net(states_t).gather(1, actions_t)
        with torch.no_grad():
            next_q = self.target_net(next_states_t).max(dim=1, keepdim=True).values
            target_q = rewards_t + self.gamma * (1.0 - terminateds_t) * next_q

        loss = self.loss_fn(current_q, target_q)
        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        return loss.item()
```

En el código anterior, la función `_learn()` realiza una actualización de aprendizaje del agente DQN después de cada paso del episodio. En MountainCar, aprende qué acción conviene tomar según la posición y velocidad actuales del automóvil. Cada experiencia tiene la forma: (**estado**, **acción**, **recompensa**, **siguiente_estado**, **terminado**), con la cual se obtiene un mini-lote aleatorio. A esto se adiciona:

- **Cálculo de valor Q actual “current_q”:** La red `q_net` calcula un valor para cada acción posible. `Gather()` selecciona únicamente el valor correspondiente a la acción que realmente se ejecutó; es decir, se estima $Q(s,a)$.
- **Cálculo del mejor valor futuro:** La red objetivo `target_net` calcula los valores de las acciones posibles en el siguiente estado y se toma el mayor:

$$\max_{a'} Q_{\text{target}}(s', a')$$

- **Construcción del objetivo de Bellman:** Si el auto alcanza la bandera, `terminateds_t` vale 1 y se elimina el valor futuro. Si el episodio termina por el límite de tiempo, `terminateds_t` vale 0, por lo que todavía se permite estimar un valor futuro. El objetivo es:

$$y = r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a')$$

- Se calcula el error y se actual `q_net`.

En resumen, la red compara su estimación actual con el objetivo de Bellman, calcula el error y ajusta sus pesos mediante descenso del gradiente. En conjunto `_learn()` hace que el agente corrija progresivamente sus estimaciones sobre qué tan buenas son las acciones de acelerar a la izquierda, no acelerar o acelerar a la derecha en cada estado de MountainCar.

Una vez resueltos estos dos ejercicios, se procede a entrenar el modelo de la red con 1.000 episodios para verificar el comportamiento. Esto se realiza con el comando: **`uv run mountaincar train dqn --episodes 1000`**

Ejecutando este comando se crea el archivo binario del agente: **`saves/dqn_mountaincar.pt`** y se generan los 1000 episodios:

```
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar train dqn --episodes 1000
Episode 880/1000 | Avg Reward: -200.00 | Epsilon: 0.0110 | Buffer: 100000
Episode 890/1000 | Avg Reward: -200.00 | Epsilon: 0.0104 | Buffer: 100000
Episode 900/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 910/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 920/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 930/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 940/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 950/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 960/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 970/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 980/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 990/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 1000/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Saved DQN agent to saves\dqn_mountaincar.pt
Training complete.
```

A continuación se muestran algunos episodios generados:

```
Episode 10/1000 | Avg Reward: -200.00 | Epsilon: 0.8604 | Buffer: 2000
Episode 20/1000 | Avg Reward: -200.00 | Epsilon: 0.8183 | Buffer: 4000
Episode 30/1000 | Avg Reward: -200.00 | Epsilon: 0.7783 | Buffer: 6000
Episode 40/1000 | Avg Reward: -200.00 | Epsilon: 0.7403 | Buffer: 8000
Episode 50/1000 | Avg Reward: -200.00 | Epsilon: 0.7041 | Buffer: 10000
Episode 60/1000 | Avg Reward: -200.00 | Epsilon: 0.6696 | Buffer: 12000
Episode 70/1000 | Avg Reward: -200.00 | Epsilon: 0.6369 | Buffer: 14000
Episode 80/1000 | Avg Reward: -200.00 | Epsilon: 0.6058 | Buffer: 16000
Episode 90/1000 | Avg Reward: -200.00 | Epsilon: 0.5762 | Buffer: 18000
```

Episode 100/1000 | Avg Reward: -200.00 | Epsilon: 0.5480 | Buffer: 20000
Episode 110/1000 | Avg Reward: -200.00 | Epsilon: 0.5212 | Buffer: 22000
Episode 120/1000 | Avg Reward: -200.00 | Epsilon: 0.4957 | Buffer: 24000
Episode 130/1000 | Avg Reward: -200.00 | Epsilon: 0.4715 | Buffer: 26000
...
pisode 840/1000 | Avg Reward: -200.00 | Epsilon: 0.0134 | Buffer: 100000
Episode 850/1000 | Avg Reward: -200.00 | Epsilon: 0.0128 | Buffer: 100000
Episode 860/1000 | Avg Reward: -200.00 | Epsilon: 0.0121 | Buffer: 100000
Episode 870/1000 | Avg Reward: -200.00 | Epsilon: 0.0115 | Buffer: 100000
Episode 880/1000 | Avg Reward: -200.00 | Epsilon: 0.0110 | Buffer: 100000
Episode 890/1000 | Avg Reward: -200.00 | Epsilon: 0.0104 | Buffer: 100000
Episode 900/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 910/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 920/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 930/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 940/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 950/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 960/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 970/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 980/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 990/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000
Episode 1000/1000 | Avg Reward: -200.00 | Epsilon: 0.0100 | Buffer: 100000

Hasta este punto se evidencia que **DQN** en MountainCar reporta una puntuación completamente plana, para siempre. No es inestable, ni mejora, se mantiene plana, exactamente en el peor valor posible, durante tantos episodios como se quieran intentar. Esto es debido a que el agente no aprendía porque la exploración epsilon-greedy elegía una acción independiente en cada paso. En **MountainCar-v0**, el auto necesita mantener acciones durante varios pasos para generar impulso. Por eso nunca alcanzaba la bandera y solo observaba recompensas -1, produciendo siempre -200.

De acuerdo a esto, vamos a intentar resolverlo de acuerdo a las pistas del Ejercicio 3 y mejorar el aprendizaje de **DQN**.

Ejercicio 3: Estrategia de exploración. Lo primero a realizar es eliminar el agente DQN que hasta el momento está creado. Esto se hace con el comando: **uv run mountaincar delete dqn**

```
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar delete dqn
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Deleted saves\dqn_mountaincar.pt
```

Luego implementamos la **política** para el agente **DQN** en la cual se incluya la estrategia de exploración. En la función **select_action()** se implementa la política **epsilon-greedy** para decidir qué acción tomar en **MountainCar**. Esta **política** será de la siguiente manera:

- **Exploración con acciones sostenidas:** Cuando **deterministic=False**, primero verifica si existe una acción exploratoria activa. Esto hace que el automóvil repita la misma acción durante varios pasos. Es importante en MountainCar porque necesita mantener una dirección durante cierto tiempo para ganar impulso y luego cambiar de dirección.
- **Inicio de una nueva exploración:** Si no hay una secuencia activa, se genera un número aleatorio y se compara con ϵ . Si se cumple, el agente:
 - Selecciona una acción aleatoria.
 - La guarda en `_exploration_action`
 - Establece cuántos pasos continuará repitiéndola.
 - Devuelve esa acción.

Al inicio del entrenamiento, ϵ es alto y el agente explora mucho. Después de cada episodio disminuye.

- **Explotación usando la red neuronal:** Si no debe explotar, la red **q_net** evalúa el estado actual. La red produce un valor Q para cada acción posible. El agente escoge la acción con el valor más alto.

$$a = \arg \max_a Q(s, a)$$

- **Modo determinista:** Cuando **deterministic=True**, se omite completamente la exploración y siempre se elige la mejor acción según la red. Esto se utiliza durante la evaluación o la visualización del agente entrenado.

Lo anterior quedaría implementado de la siguiente manera:

```
class DQNAgent:
    # — policy

    def select_action(self, state: np.ndarray, *, deterministic: bool = False) -> int:
        if not deterministic:
            if self._exploration_steps_left > 0:
                self._exploration_steps_left -= 1
                return self._exploration_action # type: ignore[return-value]
            if random.random() < self.epsilon:
                self._exploration_action = random.randrange(self.action_dim)
                self._exploration_steps_left = self._exploration_steps - 1
                return self._exploration_action
        with torch.no_grad():
            t = torch.as_tensor(state, dtype=torch.float32, device=self.device).unsqueeze(0)
            return int(self.q_net(t).argmax(dim=1).item())
```

Además, se añadiría el **hiperparámetro** `exploration_steps` para definir los pasos de exploración del agente:

```
HPARAMS = (
    "env_id",
    "lr",
    "gamma",
    "epsilon_end",
    "epsilon_decay",
    "exploration_steps",
    "batch_size",
    "buffer_capacity",
    "target_update_freq",
    "hidden",
)
```

Ya realizados estos ajustes se vuelve a crear el nuevo agente y sin entrenamiento. Esto lo realizamos con el comando: **`uv run mountaincar init dqn`**

```
(ip_py314) PS D:\MaestriaEnIA\Semestre2\Ciclo2\Code\RL\repo_taller1\mountain_car> uv run mountaincar init dqn
warning: `VIRTUAL_ENV=C:\PythonFiles\envs\ip_py314` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active
environment instead
Saved DQN agent to saves\dqn_mountaincar.pt
Initialized dqn agent.
```

Ejecutando este comando se crea el archivo binario del agente: **`saves/dqn_mountaincar.pt`**. Con la creación de la nueva política y la adición del hiperparámetro de pasos de exploración, vamos a volver a entrenar el modelo con 2.500 episodios: **`uv run mountaincar train dqn --episodes 2500`**

El nuevo detalle de los episodios es el siguiente:

Episode 10/2500	Avg Reward: -181.10	Epsilon: 0.9511	Buffer: 1811
Episode 20/2500	Avg Reward: -200.00	Epsilon: 0.9046	Buffer: 3811
Episode 30/2500	Avg Reward: -200.00	Epsilon: 0.8604	Buffer: 5811
Episode 40/2500	Avg Reward: -200.00	Epsilon: 0.8183	Buffer: 7811
Episode 50/2500	Avg Reward: -188.00	Epsilon: 0.7783	Buffer: 9691
Episode 60/2500	Avg Reward: -186.90	Epsilon: 0.7403	Buffer: 11560
Episode 70/2500	Avg Reward: -195.60	Epsilon: 0.7041	Buffer: 13516
Episode 80/2500	Avg Reward: -193.90	Epsilon: 0.6696	Buffer: 15455
Episode 90/2500	Avg Reward: -186.40	Epsilon: 0.6369	Buffer: 17319
Episode 100/2500	Avg Reward: -200.00	Epsilon: 0.6058	Buffer: 19319
Episode 110/2500	Avg Reward: -200.00	Epsilon: 0.5762	Buffer: 21319
Episode 120/2500	Avg Reward: -184.00	Epsilon: 0.5480	Buffer: 23159
Episode 130/2500	Avg Reward: -188.00	Epsilon: 0.5212	Buffer: 25039
Episode 140/2500	Avg Reward: -196.90	Epsilon: 0.4957	Buffer: 27008
Episode 150/2500	Avg Reward: -194.20	Epsilon: 0.4715	Buffer: 28950
Episode 160/2500	Avg Reward: -191.20	Epsilon: 0.4484	Buffer: 30862
Episode 170/2500	Avg Reward: -198.00	Epsilon: 0.4265	Buffer: 32842
Episode 180/2500	Avg Reward: -196.50	Epsilon: 0.4057	Buffer: 34807
Episode 190/2500	Avg Reward: -181.90	Epsilon: 0.3858	Buffer: 36626
Episode 200/2500	Avg Reward: -200.00	Epsilon: 0.3670	Buffer: 38626
Episode 210/2500	Avg Reward: -196.90	Epsilon: 0.3490	Buffer: 40595
Episode 220/2500	Avg Reward: -192.60	Epsilon: 0.3320	Buffer: 42521
Episode 230/2500	Avg Reward: -200.00	Epsilon: 0.3157	Buffer: 44521
Episode 240/2500	Avg Reward: -185.10	Epsilon: 0.3003	Buffer: 46372
Episode 250/2500	Avg Reward: -193.00	Epsilon: 0.2856	Buffer: 48302
Episode 260/2500	Avg Reward: -191.20	Epsilon: 0.2716	Buffer: 50214
Episode 270/2500	Avg Reward: -193.40	Epsilon: 0.2584	Buffer: 52148
Episode 280/2500	Avg Reward: -194.30	Epsilon: 0.2457	Buffer: 54091
Episode 290/2500	Avg Reward: -199.90	Epsilon: 0.2337	Buffer: 56090
Episode 300/2500	Avg Reward: -200.00	Epsilon: 0.2223	Buffer: 58090
Episode 310/2500	Avg Reward: -195.40	Epsilon: 0.2114	Buffer: 60044
Episode 320/2500	Avg Reward: -199.20	Epsilon: 0.2011	Buffer: 62036
Episode 330/2500	Avg Reward: -195.60	Epsilon: 0.1913	Buffer: 63992
Episode 340/2500	Avg Reward: -195.40	Epsilon: 0.1819	Buffer: 65946
Episode 350/2500	Avg Reward: -200.00	Epsilon: 0.1730	Buffer: 67946
Episode 360/2500	Avg Reward: -199.00	Epsilon: 0.1646	Buffer: 69936
Episode 370/2500	Avg Reward: -192.90	Epsilon: 0.1565	Buffer: 71865
Episode 380/2500	Avg Reward: -200.00	Epsilon: 0.1489	Buffer: 73865
Episode 390/2500	Avg Reward: -197.10	Epsilon: 0.1416	Buffer: 75836
Episode 400/2500	Avg Reward: -187.70	Epsilon: 0.1347	Buffer: 77713
Episode 410/2500	Avg Reward: -199.40	Epsilon: 0.1281	Buffer: 79707
Episode 420/2500	Avg Reward: -192.90	Epsilon: 0.1218	Buffer: 81636
Episode 430/2500	Avg Reward: -191.30	Epsilon: 0.1159	Buffer: 83549
Episode 440/2500	Avg Reward: -192.40	Epsilon: 0.1102	Buffer: 85473
Episode 450/2500	Avg Reward: -185.00	Epsilon: 0.1048	Buffer: 87323
Episode 460/2500	Avg Reward: -192.00	Epsilon: 0.0997	Buffer: 89243

Episode 470/2500	Avg Reward: -194.50	Epsilon: 0.0948	Buffer: 91188
Episode 480/2500	Avg Reward: -193.90	Epsilon: 0.0902	Buffer: 93127
Episode 490/2500	Avg Reward: -200.00	Epsilon: 0.0858	Buffer: 95127
Episode 500/2500	Avg Reward: -199.20	Epsilon: 0.0816	Buffer: 97119
Episode 510/2500	Avg Reward: -194.80	Epsilon: 0.0776	Buffer: 99067
Episode 520/2500	Avg Reward: -189.10	Epsilon: 0.0738	Buffer: 100000
Episode 530/2500	Avg Reward: -189.30	Epsilon: 0.0702	Buffer: 100000
Episode 540/2500	Avg Reward: -200.00	Epsilon: 0.0668	Buffer: 100000
Episode 550/2500	Avg Reward: -185.70	Epsilon: 0.0635	Buffer: 100000
Episode 560/2500	Avg Reward: -197.10	Epsilon: 0.0604	Buffer: 100000
Episode 570/2500	Avg Reward: -193.30	Epsilon: 0.0574	Buffer: 100000
Episode 580/2500	Avg Reward: -190.30	Epsilon: 0.0546	Buffer: 100000
Episode 590/2500	Avg Reward: -196.50	Epsilon: 0.0520	Buffer: 100000
Episode 600/2500	Avg Reward: -198.20	Epsilon: 0.0494	Buffer: 100000
Episode 610/2500	Avg Reward: -186.50	Epsilon: 0.0470	Buffer: 100000
Episode 620/2500	Avg Reward: -187.20	Epsilon: 0.0447	Buffer: 100000
Episode 630/2500	Avg Reward: -179.80	Epsilon: 0.0425	Buffer: 100000
Episode 640/2500	Avg Reward: -184.00	Epsilon: 0.0404	Buffer: 100000
Episode 650/2500	Avg Reward: -190.30	Epsilon: 0.0385	Buffer: 100000
Episode 660/2500	Avg Reward: -190.70	Epsilon: 0.0366	Buffer: 100000
Episode 670/2500	Avg Reward: -185.00	Epsilon: 0.0348	Buffer: 100000
Episode 680/2500	Avg Reward: -178.80	Epsilon: 0.0331	Buffer: 100000
Episode 690/2500	Avg Reward: -180.80	Epsilon: 0.0315	Buffer: 100000
Episode 700/2500	Avg Reward: -178.90	Epsilon: 0.0299	Buffer: 100000
Episode 710/2500	Avg Reward: -186.80	Epsilon: 0.0285	Buffer: 100000
Episode 720/2500	Avg Reward: -182.20	Epsilon: 0.0271	Buffer: 100000
Episode 730/2500	Avg Reward: -180.70	Epsilon: 0.0258	Buffer: 100000
Episode 740/2500	Avg Reward: -186.90	Epsilon: 0.0245	Buffer: 100000
Episode 750/2500	Avg Reward: -174.90	Epsilon: 0.0233	Buffer: 100000
Episode 760/2500	Avg Reward: -179.00	Epsilon: 0.0222	Buffer: 100000
Episode 770/2500	Avg Reward: -160.20	Epsilon: 0.0211	Buffer: 100000
Episode 780/2500	Avg Reward: -166.70	Epsilon: 0.0200	Buffer: 100000
Episode 790/2500	Avg Reward: -181.10	Epsilon: 0.0191	Buffer: 100000
Episode 800/2500	Avg Reward: -185.10	Epsilon: 0.0181	Buffer: 100000
Episode 810/2500	Avg Reward: -174.10	Epsilon: 0.0172	Buffer: 100000
Episode 820/2500	Avg Reward: -169.30	Epsilon: 0.0164	Buffer: 100000
Episode 830/2500	Avg Reward: -152.20	Epsilon: 0.0156	Buffer: 100000
Episode 840/2500	Avg Reward: -166.60	Epsilon: 0.0148	Buffer: 100000
Episode 850/2500	Avg Reward: -170.80	Epsilon: 0.0141	Buffer: 100000
Episode 860/2500	Avg Reward: -172.40	Epsilon: 0.0134	Buffer: 100000
Episode 870/2500	Avg Reward: -153.00	Epsilon: 0.0128	Buffer: 100000
Episode 880/2500	Avg Reward: -144.10	Epsilon: 0.0121	Buffer: 100000
Episode 890/2500	Avg Reward: -152.40	Epsilon: 0.0115	Buffer: 100000
Episode 900/2500	Avg Reward: -148.40	Epsilon: 0.0110	Buffer: 100000
Episode 910/2500	Avg Reward: -145.70	Epsilon: 0.0104	Buffer: 100000
Episode 920/2500	Avg Reward: -147.90	Epsilon: 0.0100	Buffer: 100000

Episode 930/2500	Avg Reward: -122.60	Epsilon: 0.0100	Buffer: 100000
Episode 940/2500	Avg Reward: -140.20	Epsilon: 0.0100	Buffer: 100000
Episode 950/2500	Avg Reward: -140.50	Epsilon: 0.0100	Buffer: 100000
Episode 960/2500	Avg Reward: -133.80	Epsilon: 0.0100	Buffer: 100000
Episode 970/2500	Avg Reward: -112.00	Epsilon: 0.0100	Buffer: 100000
Episode 980/2500	Avg Reward: -125.50	Epsilon: 0.0100	Buffer: 100000
Episode 990/2500	Avg Reward: -119.00	Epsilon: 0.0100	Buffer: 100000
Episode 1000/2500	Avg Reward: -123.60	Epsilon: 0.0100	Buffer: 100000
Episode 1010/2500	Avg Reward: -139.50	Epsilon: 0.0100	Buffer: 100000
Episode 1020/2500	Avg Reward: -118.60	Epsilon: 0.0100	Buffer: 100000
Episode 1030/2500	Avg Reward: -140.50	Epsilon: 0.0100	Buffer: 100000
Episode 1040/2500	Avg Reward: -154.90	Epsilon: 0.0100	Buffer: 100000
Episode 1050/2500	Avg Reward: -131.30	Epsilon: 0.0100	Buffer: 100000
Episode 1060/2500	Avg Reward: -136.20	Epsilon: 0.0100	Buffer: 100000
Episode 1070/2500	Avg Reward: -121.40	Epsilon: 0.0100	Buffer: 100000
Episode 1080/2500	Avg Reward: -115.60	Epsilon: 0.0100	Buffer: 100000
Episode 1090/2500	Avg Reward: -134.40	Epsilon: 0.0100	Buffer: 100000
Episode 1100/2500	Avg Reward: -102.70	Epsilon: 0.0100	Buffer: 100000
Episode 1110/2500	Avg Reward: -144.70	Epsilon: 0.0100	Buffer: 100000
Episode 1120/2500	Avg Reward: -144.30	Epsilon: 0.0100	Buffer: 100000
Episode 1130/2500	Avg Reward: -124.00	Epsilon: 0.0100	Buffer: 100000
Episode 1140/2500	Avg Reward: -142.10	Epsilon: 0.0100	Buffer: 100000
Episode 1150/2500	Avg Reward: -133.80	Epsilon: 0.0100	Buffer: 100000
Episode 1160/2500	Avg Reward: -131.50	Epsilon: 0.0100	Buffer: 100000
Episode 1170/2500	Avg Reward: -136.20	Epsilon: 0.0100	Buffer: 100000
Episode 1180/2500	Avg Reward: -130.20	Epsilon: 0.0100	Buffer: 100000
Episode 1190/2500	Avg Reward: -136.20	Epsilon: 0.0100	Buffer: 100000
Episode 1200/2500	Avg Reward: -124.50	Epsilon: 0.0100	Buffer: 100000
Episode 1210/2500	Avg Reward: -135.60	Epsilon: 0.0100	Buffer: 100000
Episode 1220/2500	Avg Reward: -144.20	Epsilon: 0.0100	Buffer: 100000
Episode 1230/2500	Avg Reward: -113.50	Epsilon: 0.0100	Buffer: 100000
Episode 1240/2500	Avg Reward: -124.80	Epsilon: 0.0100	Buffer: 100000
Episode 1250/2500	Avg Reward: -139.20	Epsilon: 0.0100	Buffer: 100000
Episode 1260/2500	Avg Reward: -125.10	Epsilon: 0.0100	Buffer: 100000
Episode 1270/2500	Avg Reward: -153.40	Epsilon: 0.0100	Buffer: 100000
Episode 1280/2500	Avg Reward: -137.60	Epsilon: 0.0100	Buffer: 100000
Episode 1290/2500	Avg Reward: -138.40	Epsilon: 0.0100	Buffer: 100000
Episode 1300/2500	Avg Reward: -139.70	Epsilon: 0.0100	Buffer: 100000
Episode 1310/2500	Avg Reward: -153.80	Epsilon: 0.0100	Buffer: 100000
Episode 1320/2500	Avg Reward: -121.40	Epsilon: 0.0100	Buffer: 100000
Episode 1330/2500	Avg Reward: -139.20	Epsilon: 0.0100	Buffer: 100000
Episode 1340/2500	Avg Reward: -146.80	Epsilon: 0.0100	Buffer: 100000
Episode 1350/2500	Avg Reward: -144.90	Epsilon: 0.0100	Buffer: 100000
Episode 1360/2500	Avg Reward: -116.60	Epsilon: 0.0100	Buffer: 100000
Episode 1370/2500	Avg Reward: -143.60	Epsilon: 0.0100	Buffer: 100000
Episode 1380/2500	Avg Reward: -154.80	Epsilon: 0.0100	Buffer: 100000

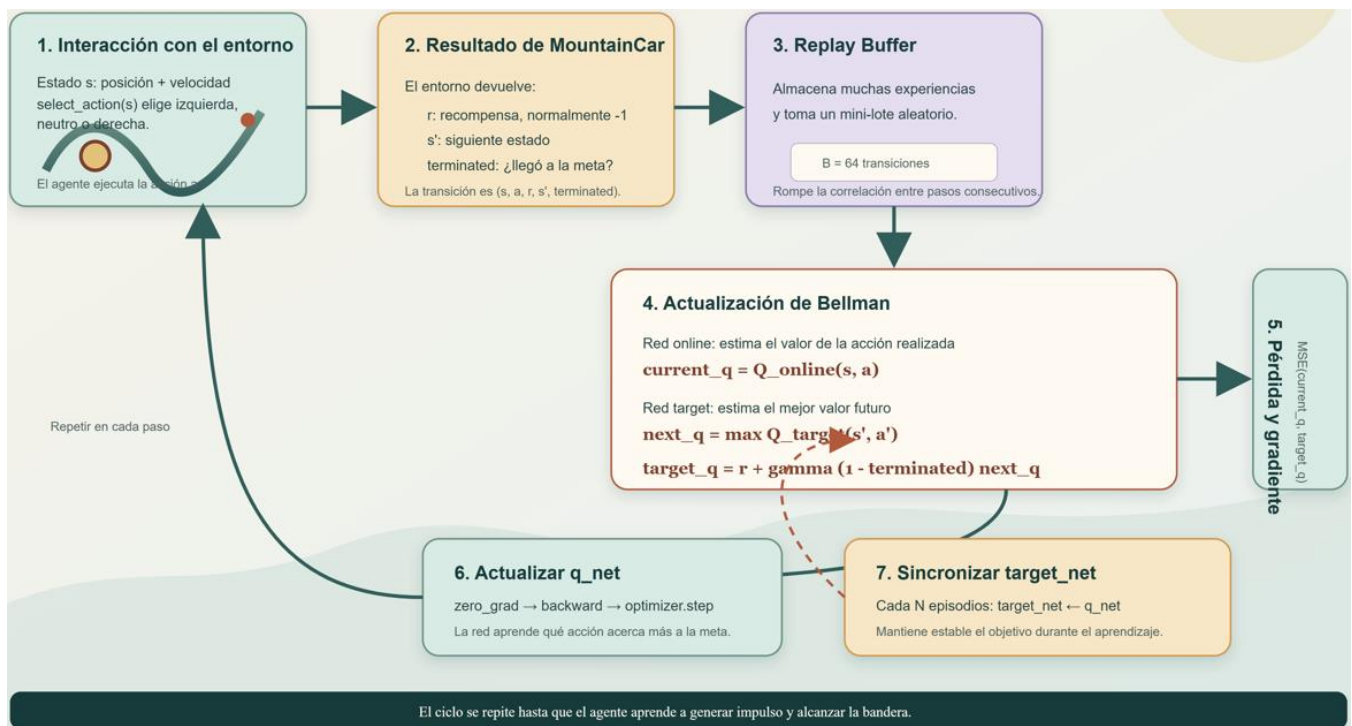
Episode 1390/2500	Avg Reward: -141.20	Epsilon: 0.0100	Buffer: 100000
Episode 1400/2500	Avg Reward: -125.50	Epsilon: 0.0100	Buffer: 100000
Episode 1410/2500	Avg Reward: -137.30	Epsilon: 0.0100	Buffer: 100000
Episode 1420/2500	Avg Reward: -131.20	Epsilon: 0.0100	Buffer: 100000
Episode 1430/2500	Avg Reward: -129.70	Epsilon: 0.0100	Buffer: 100000
Episode 1440/2500	Avg Reward: -128.00	Epsilon: 0.0100	Buffer: 100000
Episode 1450/2500	Avg Reward: -139.50	Epsilon: 0.0100	Buffer: 100000
Episode 1460/2500	Avg Reward: -139.10	Epsilon: 0.0100	Buffer: 100000
Episode 1470/2500	Avg Reward: -151.10	Epsilon: 0.0100	Buffer: 100000
Episode 1480/2500	Avg Reward: -149.80	Epsilon: 0.0100	Buffer: 100000
Episode 1490/2500	Avg Reward: -129.00	Epsilon: 0.0100	Buffer: 100000
Episode 1500/2500	Avg Reward: -132.60	Epsilon: 0.0100	Buffer: 100000
Episode 1510/2500	Avg Reward: -128.60	Epsilon: 0.0100	Buffer: 100000
Episode 1520/2500	Avg Reward: -135.20	Epsilon: 0.0100	Buffer: 100000
Episode 1530/2500	Avg Reward: -118.40	Epsilon: 0.0100	Buffer: 100000
Episode 1540/2500	Avg Reward: -149.30	Epsilon: 0.0100	Buffer: 100000
Episode 1550/2500	Avg Reward: -125.30	Epsilon: 0.0100	Buffer: 100000
Episode 1560/2500	Avg Reward: -124.60	Epsilon: 0.0100	Buffer: 100000
Episode 1570/2500	Avg Reward: -134.70	Epsilon: 0.0100	Buffer: 100000
Episode 1580/2500	Avg Reward: -132.80	Epsilon: 0.0100	Buffer: 100000
Episode 1590/2500	Avg Reward: -141.50	Epsilon: 0.0100	Buffer: 100000
Episode 1600/2500	Avg Reward: -125.10	Epsilon: 0.0100	Buffer: 100000
Episode 1610/2500	Avg Reward: -120.10	Epsilon: 0.0100	Buffer: 100000
Episode 1620/2500	Avg Reward: -121.70	Epsilon: 0.0100	Buffer: 100000
Episode 1630/2500	Avg Reward: -124.70	Epsilon: 0.0100	Buffer: 100000
Episode 1640/2500	Avg Reward: -143.40	Epsilon: 0.0100	Buffer: 100000
Episode 1650/2500	Avg Reward: -134.10	Epsilon: 0.0100	Buffer: 100000
Episode 1660/2500	Avg Reward: -134.90	Epsilon: 0.0100	Buffer: 100000
Episode 1670/2500	Avg Reward: -129.10	Epsilon: 0.0100	Buffer: 100000
Episode 1680/2500	Avg Reward: -118.40	Epsilon: 0.0100	Buffer: 100000
Episode 1690/2500	Avg Reward: -155.50	Epsilon: 0.0100	Buffer: 100000
Episode 1700/2500	Avg Reward: -125.40	Epsilon: 0.0100	Buffer: 100000
Episode 1710/2500	Avg Reward: -128.70	Epsilon: 0.0100	Buffer: 100000
Episode 1720/2500	Avg Reward: -129.90	Epsilon: 0.0100	Buffer: 100000
Episode 1730/2500	Avg Reward: -134.10	Epsilon: 0.0100	Buffer: 100000
Episode 1740/2500	Avg Reward: -136.10	Epsilon: 0.0100	Buffer: 100000
Episode 1750/2500	Avg Reward: -125.00	Epsilon: 0.0100	Buffer: 100000
Episode 1760/2500	Avg Reward: -128.60	Epsilon: 0.0100	Buffer: 100000
Episode 1770/2500	Avg Reward: -134.90	Epsilon: 0.0100	Buffer: 100000
Episode 1780/2500	Avg Reward: -132.10	Epsilon: 0.0100	Buffer: 100000
Episode 1790/2500	Avg Reward: -122.10	Epsilon: 0.0100	Buffer: 100000
Episode 1800/2500	Avg Reward: -114.70	Epsilon: 0.0100	Buffer: 100000
Episode 1810/2500	Avg Reward: -150.40	Epsilon: 0.0100	Buffer: 100000
Episode 1820/2500	Avg Reward: -122.10	Epsilon: 0.0100	Buffer: 100000
Episode 1830/2500	Avg Reward: -128.60	Epsilon: 0.0100	Buffer: 100000
Episode 1840/2500	Avg Reward: -123.80	Epsilon: 0.0100	Buffer: 100000

Episode 1850/2500	Avg Reward: -123.00	Epsilon: 0.0100	Buffer: 100000
Episode 1860/2500	Avg Reward: -127.50	Epsilon: 0.0100	Buffer: 100000
Episode 1870/2500	Avg Reward: -139.00	Epsilon: 0.0100	Buffer: 100000
Episode 1880/2500	Avg Reward: -149.90	Epsilon: 0.0100	Buffer: 100000
Episode 1890/2500	Avg Reward: -133.70	Epsilon: 0.0100	Buffer: 100000
Episode 1900/2500	Avg Reward: -152.10	Epsilon: 0.0100	Buffer: 100000
Episode 1910/2500	Avg Reward: -131.80	Epsilon: 0.0100	Buffer: 100000
Episode 1920/2500	Avg Reward: -138.70	Epsilon: 0.0100	Buffer: 100000
Episode 1930/2500	Avg Reward: -131.40	Epsilon: 0.0100	Buffer: 100000
Episode 1940/2500	Avg Reward: -137.30	Epsilon: 0.0100	Buffer: 100000
Episode 1950/2500	Avg Reward: -130.10	Epsilon: 0.0100	Buffer: 100000
Episode 1960/2500	Avg Reward: -146.10	Epsilon: 0.0100	Buffer: 100000
Episode 1970/2500	Avg Reward: -144.80	Epsilon: 0.0100	Buffer: 100000
Episode 1980/2500	Avg Reward: -147.10	Epsilon: 0.0100	Buffer: 100000
Episode 1990/2500	Avg Reward: -140.50	Epsilon: 0.0100	Buffer: 100000
Episode 2000/2500	Avg Reward: -123.20	Epsilon: 0.0100	Buffer: 100000
Episode 2010/2500	Avg Reward: -158.80	Epsilon: 0.0100	Buffer: 100000
Episode 2020/2500	Avg Reward: -143.50	Epsilon: 0.0100	Buffer: 100000
Episode 2030/2500	Avg Reward: -122.30	Epsilon: 0.0100	Buffer: 100000
Episode 2040/2500	Avg Reward: -130.70	Epsilon: 0.0100	Buffer: 100000
Episode 2050/2500	Avg Reward: -161.20	Epsilon: 0.0100	Buffer: 100000
Episode 2060/2500	Avg Reward: -107.40	Epsilon: 0.0100	Buffer: 100000
Episode 2070/2500	Avg Reward: -115.80	Epsilon: 0.0100	Buffer: 100000
Episode 2080/2500	Avg Reward: -107.40	Epsilon: 0.0100	Buffer: 100000
Episode 2090/2500	Avg Reward: -127.00	Epsilon: 0.0100	Buffer: 100000
Episode 2100/2500	Avg Reward: -115.10	Epsilon: 0.0100	Buffer: 100000
Episode 2110/2500	Avg Reward: -125.80	Epsilon: 0.0100	Buffer: 100000
Episode 2120/2500	Avg Reward: -132.00	Epsilon: 0.0100	Buffer: 100000
Episode 2130/2500	Avg Reward: -123.20	Epsilon: 0.0100	Buffer: 100000
Episode 2140/2500	Avg Reward: -124.70	Epsilon: 0.0100	Buffer: 100000
Episode 2150/2500	Avg Reward: -115.20	Epsilon: 0.0100	Buffer: 100000
Episode 2160/2500	Avg Reward: -124.50	Epsilon: 0.0100	Buffer: 100000
Episode 2170/2500	Avg Reward: -131.10	Epsilon: 0.0100	Buffer: 100000
Episode 2180/2500	Avg Reward: -135.90	Epsilon: 0.0100	Buffer: 100000
Episode 2190/2500	Avg Reward: -120.20	Epsilon: 0.0100	Buffer: 100000
Episode 2200/2500	Avg Reward: -135.40	Epsilon: 0.0100	Buffer: 100000
Episode 2210/2500	Avg Reward: -127.20	Epsilon: 0.0100	Buffer: 100000
Episode 2220/2500	Avg Reward: -147.40	Epsilon: 0.0100	Buffer: 100000
Episode 2230/2500	Avg Reward: -132.00	Epsilon: 0.0100	Buffer: 100000
Episode 2240/2500	Avg Reward: -133.50	Epsilon: 0.0100	Buffer: 100000
Episode 2250/2500	Avg Reward: -142.70	Epsilon: 0.0100	Buffer: 100000
Episode 2260/2500	Avg Reward: -131.70	Epsilon: 0.0100	Buffer: 100000
Episode 2270/2500	Avg Reward: -132.20	Epsilon: 0.0100	Buffer: 100000
Episode 2280/2500	Avg Reward: -133.10	Epsilon: 0.0100	Buffer: 100000
Episode 2290/2500	Avg Reward: -137.90	Epsilon: 0.0100	Buffer: 100000
Episode 2300/2500	Avg Reward: -130.80	Epsilon: 0.0100	Buffer: 100000

Episode 2310/2500	Avg Reward: -121.60	Epsilon: 0.0100	Buffer: 100000
Episode 2320/2500	Avg Reward: -123.50	Epsilon: 0.0100	Buffer: 100000
Episode 2330/2500	Avg Reward: -112.80	Epsilon: 0.0100	Buffer: 100000
Episode 2340/2500	Avg Reward: -123.80	Epsilon: 0.0100	Buffer: 100000
Episode 2350/2500	Avg Reward: -125.80	Epsilon: 0.0100	Buffer: 100000
Episode 2360/2500	Avg Reward: -136.90	Epsilon: 0.0100	Buffer: 100000
Episode 2370/2500	Avg Reward: -151.80	Epsilon: 0.0100	Buffer: 100000
Episode 2380/2500	Avg Reward: -132.00	Epsilon: 0.0100	Buffer: 100000
Episode 2390/2500	Avg Reward: -127.60	Epsilon: 0.0100	Buffer: 100000
Episode 2400/2500	Avg Reward: -117.80	Epsilon: 0.0100	Buffer: 100000
Episode 2410/2500	Avg Reward: -141.70	Epsilon: 0.0100	Buffer: 100000
Episode 2420/2500	Avg Reward: -140.50	Epsilon: 0.0100	Buffer: 100000
Episode 2430/2500	Avg Reward: -129.10	Epsilon: 0.0100	Buffer: 100000
Episode 2440/2500	Avg Reward: -114.50	Epsilon: 0.0100	Buffer: 100000
Episode 2450/2500	Avg Reward: -132.30	Epsilon: 0.0100	Buffer: 100000
Episode 2460/2500	Avg Reward: -104.10	Epsilon: 0.0100	Buffer: 100000
Episode 2470/2500	Avg Reward: -125.60	Epsilon: 0.0100	Buffer: 100000
Episode 2480/2500	Avg Reward: -117.90	Epsilon: 0.0100	Buffer: 100000
Episode 2490/2500	Avg Reward: -119.80	Epsilon: 0.0100	Buffer: 100000
Episode 2500/2500	Avg Reward: -132.20	Epsilon: 0.0100	Buffer: 100000

Cada ciclo de entrenamiento dado por los episodios pueden entenderse de la siguiente manera:

Figura 6. Ciclo de entrenamiento de agente DQN.



Ahora se procede a evaluar y a imprimir la información consolidada del resultado obtenido con el comando: ***uv run mountaincar load dqn --eval***

```
DQN agent for MountainCar-v0
Episodes trained : 2500
Network params   : 17,283
Epsilon          : 0.0100
LR / Gamma       : 0.001 / 0.99
Batch size       : 64
Target update    : every 10 episodes
Device           : cpu

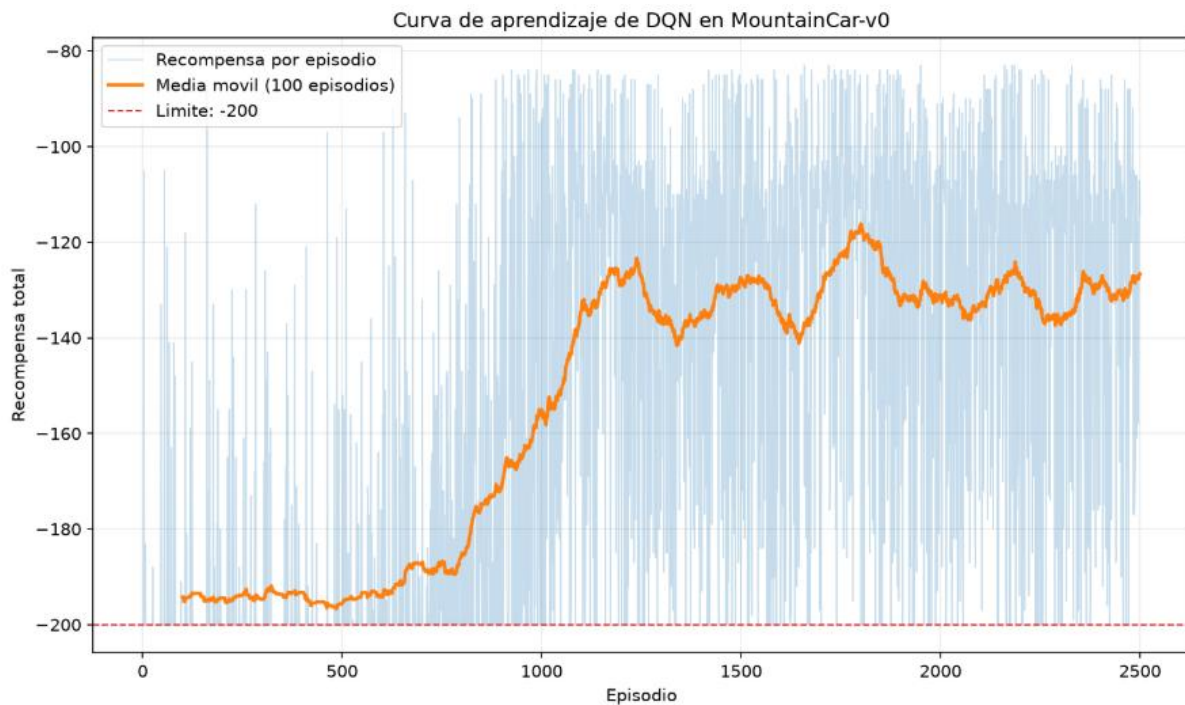
Evaluating (10 episodes) ...
Mean reward: -99.90 +/- 7.01
Reached the flag: 10/10 episodes
```

A partir de los resultados obtenidos en el entrenamiento y en la evaluación, se evidencia que con la **implementación de la política** en la función **select_action()**, el agente DQN aprendió correctamente la tarea y alcanzó su objetivo con éxito al lograr que el auto llegue a la bandera en 10/10 episodios. Además es importante resaltar que:

- Cuando Recompensa media de -99.90: en MountainCar se recibe -1 por cada paso. Por tanto, el agente tarda aproximadamente 100 pasos en alcanzar la meta, frente al límite de 200 pasos.
- Comparación: una política que nunca alcanza la bandera obtiene cerca de -200. El resultado de -99.90 representa una mejora importante.
- Baja variabilidad: ± 7.01 indica un comportamiento relativamente estable entre episodios.
- Epsilon 0.0100: durante el entrenamiento prácticamente dejó de explorar y pasó a explotar lo aprendido. La evaluación refleja principalmente la política aprendida.
- Gamma 0.99: el agente valora las recompensas futuras, algo importante porque en MountainCar debe ejecutar una secuencia de movimientos para generar impulso.
- Target cada 10 episodios: la red objetivo se actualiza periódicamente, ayudando a estabilizar el aprendizaje.

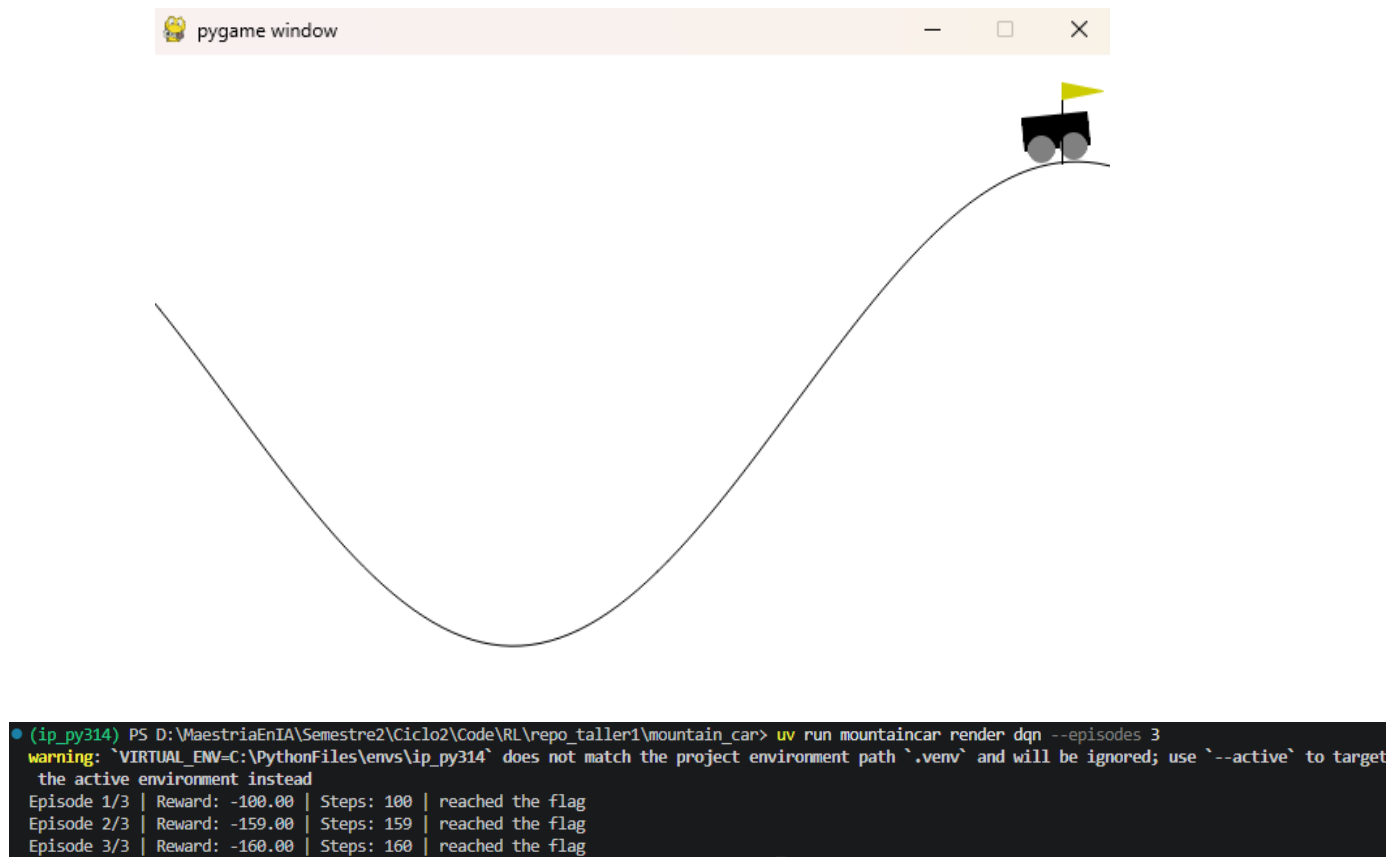
A continuación vemos la **curva de aprendizaje** del agente **DQN** a través de los 2.500 episodios, generada con el siguiente comando ejecutado desde la carpeta **mountain_car**:
.\venv\Scripts\python.exe .\graficos\graficas_dqn.py --episodes 2500 --window 100

Figura 7. Curva de aprendizaje del agente DQN



En la gráfica anterior se puede ver el resumen del comportamiento del entrenamiento, en donde inicialmente el agente no aprende, se mantiene cerca de -200, alrededor de -198 o -197 y posteriormente cuando hay más exploración de estados su aprendizaje va mejorando, hasta alcanzar recompensas medias más elevadas. **Alrededor del episodio 1.800 se puede observar el pico más alta de recompensas medias que alcanzan valores mayores a -120.**

Finalmente, vamos a reproducir episodios en una ventana gráfica a través del siguiente comando: ***uv run mountaincar render dqn --episodes 3***



En la gráfica anterior, se puede observar cómo el auto logra llegar al objetivo (la bandera en la cima de la colina derecha) en los tres episodios de prueba.

V. PASO 4: DOCUMENTACIÓN Y COMPARACIÓN DE RESULTADOS

Comparación de resultados:

MountainCar-v0 entrega una recompensa de -1 por paso y tiene un límite de 200 pasos. Por tanto, una recompensa menos negativa representa un mejor desempeño: -100 significa que el agente llegó a la bandera aproximadamente en 100 pasos, mientras que -200 indica que agotó el tiempo.

Tabla 2. *Comparación de los dos agentes*

Criterio	Q-Learning	DQN
Representación	Tabla con estados discretizados en 20 x 20 = 400 celdas	Red neuronal sobre posición y velocidad continuas
Entrenamiento de referencia	20.000 episodios	2.500 episodios

Mejor recompensa de entrenamiento	-110	Aproximadamente -83 en la curva observada
Mejor media móvil	-139.52, alrededor del episodio 4.693	Aproximadamente -118 en la curva
Evaluación disponible	Media -177.51, éxito 52/100	Media -99.90 +/- 7.01, éxito 10/10
Estabilidad en evaluación	Baja: desviación estándar 21.87	Alta: desviación estándar 7.01

Estabilidad del entrenamiento:

La curva de Q-Learning presenta una mejora progresiva, pero con oscilaciones importantes. La media móvil comienza cerca de -200, mejora hasta aproximadamente -140 y posteriormente presenta retrocesos y recuperaciones. Esto indica que la tabla aprende, pero sus actualizaciones son sensibles a la exploración y a la discretización del espacio de estados.

La curva de DQN también tiene recompensas individuales muy variables, pero su media móvil muestra una mejora más marcada: pasa de aproximadamente -195 a valores cercanos a -120 o -130. Después alcanza una meseta relativamente estable. Además, la evaluación reciente obtuvo una desviación estándar de solo 7.01, lo cual indica un comportamiento más consistente.

Velocidad de aprendizaje:

DQN aprendió una política exitosa en aproximadamente 2.500 episodios, mientras que Q-Learning requiere alrededor de 20.000 episodios según la configuración y los resultados de referencia del proyecto.

Esto favorece a DQN en términos de cantidad de episodios necesarios. La razón principal es que la red neuronal generaliza entre estados cercanos; no necesita aprender cada celda de la tabla por separado.

Sin embargo, la comparación debe interpretarse con cierta cautela, porque ambos agentes utilizan hiperparámetros y dinámicas de exploración diferentes.

Desempeño final:

El mejor resultado corresponde al DQN:

- Recompensa media: -99.90.
- Éxito: 10/10 episodios.
- Aproximadamente 100 pasos por episodio.

- Supera el umbral práctico de resolución de MountainCar, cercano a -110.

El Q-Learning también consigue aprender parcialmente:

- Mejor recompensa de entrenamiento: -110.
- Mejor media móvil: -139.52.
- En la evaluación archivada: 52/100 éxitos.
- Recompensa media de evaluación: -177.51.

Por tanto, Q-Learning descubre la estrategia de balanceo, pero no la ejecuta con la misma regularidad que DQN.

Ventajas y limitaciones:

Q-Learning ventajas:

Es más sencillo de entender e implementar.

La tabla permite inspeccionar directamente los valores Q.

Tiene un costo computacional bajo.

No requiere arquitectura neuronal, optimizador ni retropropagación.

Q-Learning limitaciones:

Necesita discretizar los estados continuos.

La calidad depende mucho del número de divisiones.

Con pocas divisiones pierde precisión; con muchas, la tabla crece rápidamente.

Presenta menor generalización entre estados parecidos.

Requiere más episodios para obtener un comportamiento consistente.

DQN ventajas:

Trabaja directamente con los estados continuos de MountainCar.

Generaliza entre estados similares.

Aprende con menos episodios en esta experiencia.

Su combinación de Replay Buffer y target_net mejora la estabilidad del aprendizaje.

Logró una tasa de éxito del 100% en la evaluación reciente.

DQN limitaciones:

Es más complejo de implementar y depurar.

Requiere elegir arquitectura, tasa de aprendizaje, tamaño de lote y frecuencia de actualización de la red objetivo.

Las recompensas individuales son muy ruidosas.

Puede fallar completamente si la estrategia de exploración no genera secuencias sostenidas de acciones.

Sus resultados pueden variar más entre ejecuciones y semillas aleatorias.

Conclusión global:

La primera evaluación de DQN mostró una recompensa de -200 y éxito 0%. Esa ejecución fue fallida y no representa el resultado más reciente, ya que posteriormente el entrenamiento de 2.500 episodios produjo:

Mean reward: -99.90 +/- 7.01

Reached the flag: 10/10 episodes

Por lo que se puede evidenciar que, DQN obtuvo el mejor resultado final y aprendió más rápido, mientras que Q-Learning ofrece una implementación más simple y transparente, pero necesita más episodios y presentó un desempeño final menos consistente en las evaluaciones disponibles.

VI. REFERENCIAS

Lapan, M. (2020). *Deep Reinforcement Learning Hands-On*. Packt.

Sutton, R., & Barto, A. (2018). *Reinforcement learning: An introduction*. MIT Press.